

Towards Monosemanticity: Decomposing Language Models With Dictionary Learning

Using a sparse autoencoder, we extract a large number of interpretable features from a one-layer transformer.

[Browse A/1 Features →](#)

[Browse All Features →](#)

AUTHORS

Trenton Bricken*, Adly Templeton*, Joshua Batson*, Brian Chen*, Adam Jermy*, Tom Conerly, Nicholas L Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, Chris Olah

AFFILIATIONS

Anthropic

PUBLISHED

Oct 4, 2023

* Core Contributor; Correspondence to colah@anthropic.com; Author contributions statement below.

Mechanistic interpretability seeks to understand neural networks by breaking them into components that are more easily understood than the whole. By understanding the function of each component, and how they interact, we hope to be able to reason about the behavior of the entire network. The first step in that program is to identify the correct components to analyze.

Unfortunately, the most natural computational unit of the neural network – the neuron itself – turns out not to be a natural unit for human understanding. This is because many neurons are *polysemantic*: they respond to mixtures of seemingly unrelated inputs. In the vision model *Inception v1*, a single neuron responds to faces of cats and fronts of cars [1]. In a small language model we discuss in this paper, a single neuron responds to a mixture of academic citations, English dialogue, HTTP requests, and Korean text. Polysemanticity makes it difficult to reason about the behavior of the network in terms of the activity of individual neurons.

One potential cause of polysemanticity is *superposition* [2], a hypothesized phenomenon where a neural network represents more independent "features" of the data than it has neurons by assigning each feature its own linear combination of neurons. If we view each feature as a vector over the neurons, then the set of features form an overcomplete linear basis for the activations of the network neurons. In our previous paper on *Toy Models of Superposition* [3], we showed that superposition can arise naturally during the course of neural network training if the set of features useful to a model are sparse in the training data. As in compressed sensing, sparsity allows a model to disambiguate which combination of features produced any given activation vector.¹

In *Toy Models of Superposition*, we described three strategies to finding a sparse and interpretable set of features if they are indeed hidden by superposition: (1) creating models without superposition, perhaps by encouraging activation sparsity; (2) using dictionary learning to find an overcomplete feature basis in a model exhibiting superposition; and (3) hybrid approaches relying on a combination of the two. Since the publication of that work, we've explored all three approaches. We eventually developed counterexamples which persuaded us that the sparse architectural approach (approach 1) was insufficient to prevent polysematicity, and that standard dictionary learning methods (approach 2) had significant issues with overfitting.

In this paper, we use a weak dictionary learning algorithm called a *sparse autoencoder* to generate *learned features* from a trained model that offer a more monosemantic unit of analysis than the model's neurons themselves. Our approach here builds on a significant amount of prior work, especially in using dictionary learning and related methods on neural network activations (e.g. [\[1\]](#)), and a more general allied literature on *disentanglement*. We also note interim reports [\[2\]](#) which independently investigated the sparse autoencoder approach in response to *Toy Models*, culminating in the recent manuscript of Cunningham *et al.* [\[3\]](#).

The goal of this paper is to provide a detailed demonstration of a sparse autoencoder compellingly succeeding at the goals of extracting interpretable features from superposition and enabling basic circuit analysis. Concretely, we take a one-layer transformer with a 512-neuron MLP layer, and decompose the MLP activations into relatively interpretable features by training sparse autoencoders on MLP activations from 8 billion data points, with expansion factors ranging from $1\times$ (512 features) to $256\times$ (131,072 features). We focus our detailed interpretability analyses on the 4,096 features learned in one run we call A/1.

This report has four major sections. In **Problem Setup**, we provide motivation for our approach and describe the transformers and sparse autoencoders we train. In **Detailed Investigations of Individual Features**, we offer an existence proof – we make the case that several features we find are functionally specific causal units which don't correspond to neurons. In **Global Analysis**, we argue that the typical feature is interpretable and that they explain a non-trivial portion of the MLP layer. Finally, in **Phenomenology** we describe several properties of our features, including feature-splitting, universality, and how they can form "finite state automata"-like systems implementing interesting behaviors.

We also provide three comprehensive visualizations of features. First, for all features from 90 learned dictionaries we present activating dataset examples and downstream logit effects. We recommend the reader begin with the **visualization of A/1**. Second, we provide a data-oriented view, showing all features active on each token of 25 texts. Finally, we coembed all 4,096 features from A/1 and all 512 features from A/0 into the plane using UMAP to allow for interactive exploration of the space of features:

This visualization only works inline on larger screens.
[View standalone version in a new window.](#)

Summary of Results

- **Sparse Autoencoders extract relatively monosemantic features.** We provide *four* different lines of evidence:

detailed investigations for a few features firing in specific contexts for which we can construct computational proxies, human analysis for a large random sample of features, automated interpretability analysis of activations for all the features learned by the autoencoder, and finally automated interpretability analysis of logit weights for all the features. Moreover, the last three analyses show that *most* learned features are interpretable. While we do not claim that our interpretations catch all aspects of features' behaviors, by constructing metrics of interpretability consistently for features and neurons, we quantitatively show their relative interpretability.

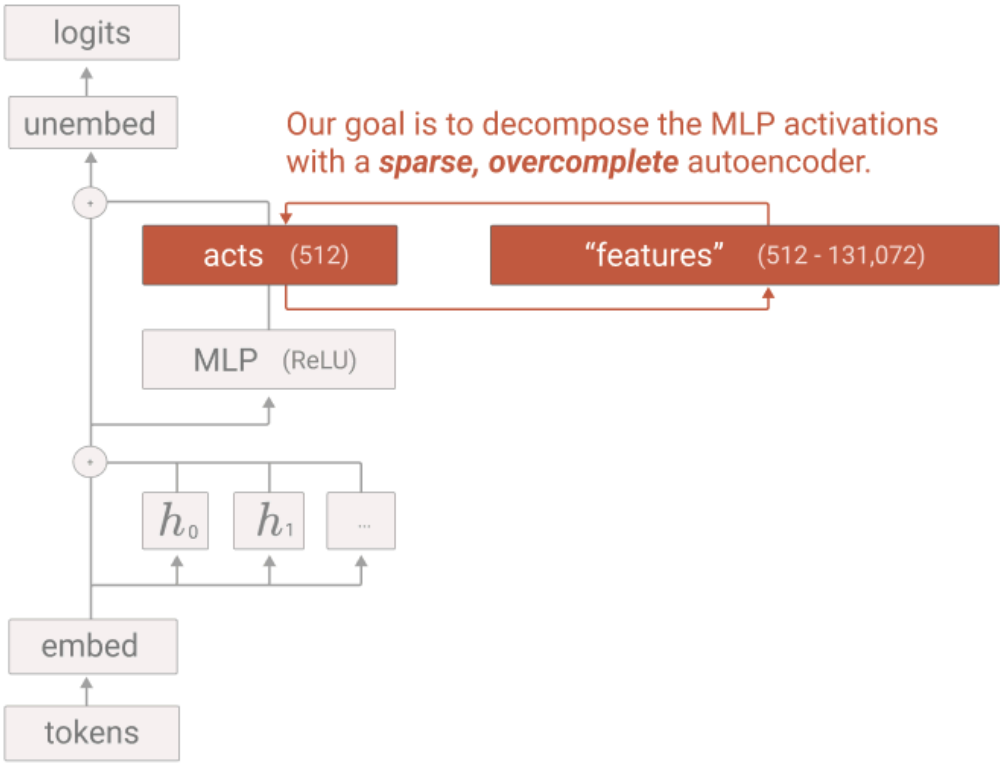
- **Sparse autoencoders produce interpretable features that are effectively invisible in the neuron basis.** We find features (e.g., one firing on Hebrew script) which are not active in any of the top dataset examples for any of the neurons.
- **Sparse autoencoder features can be used to intervene on and steer transformer generation.** For example, activating the base64 feature we study causes the model to generate base64 text, and activating the Arabic script feature we study produces Arabic text. (See discussion of pinned feature sampling in Global Analysis.)
- **Sparse autoencoders produce relatively universal features.** Sparse autoencoders applied to different transformer language models produce mostly similar features, more similar to one another than they are to their own model's neurons. (See Universality)
- **Features appear to "split" as we increase autoencoder size.** When we gradually increase the width of the autoencoder from 512 (the number of neurons) to over 131,000 (256×), we find features which naturally fit together into families. For example, one base64 feature in a small dictionary splits into three, with more subtle and yet still interpretable roles, in a larger dictionary. The different size autoencoders offer different "resolutions" for understanding the same object. (See Feature Splitting.)
- **Just 512 neurons can represent tens of thousands of features.** Despite the MLP layer being very small, we continue to find new features as we scale the sparse autoencoder.
- **Features connect in "finite-state automata"-like systems that implement complex behaviors.** For example, we find features that work together to generate valid HTML. (See "Finite State Automata".)

Problem Setup

A key challenge to our agenda of reverse engineering neural networks is the curse of dimensionality: as we study ever-larger models, the volume of the latent space representing the model's internal state that we need to interpret grows exponentially. We do not currently see a way to understand, search or enumerate such a space unless it can be decomposed into independent components, each of which we can understand on its own.

In certain limited cases, it is possible to side step these issues by rewriting neural networks in ways that don't make reference to certain hidden states. For example, in *A Mathematical Framework for Transformer Circuits* [\[1\]](#), we were able to analyze a one-layer attention-only network without addressing this problem. But this approach becomes impossible if we consider even a simple standard one-layer transformer with an MLP layer with a ReLU activation function. Understanding such a model requires us to have a way to decompose the MLP layer.

In some sense, this is the simplest language model we *profoundly don't understand*. And so it makes a natural target for our paper. We aim to take its MLP activations – the activations we can't avoid needing to decompose – and decompose them into "features":



Crucially, we decompose into *more features than there are neurons*. This is because we believe that the MLP layer likely uses superposition [\[2\]](#) to represent more features than it has neurons (and correspondingly do more useful non-linear work!). In fact, in our largest experiments we'll expand to have 256 times more features than neurons, although we'll primarily focus on a more modest 8× expansion.

	Transformer	Sparse Autoencoder
Layers	1 Attention Block 1 MLP Block (ReLU)	1 ReLU (up) 1 Linear (down)
MLP Size	512	512 (1×) – 131,072 (256×)

Dataset	The Pile [1] (100 billion tokens)	Transformer MLP Activations (8 billion samples)
Loss	Autoregressive Log-Likelihood	L2 reconstruction + L1 on hidden layer activation

In the following subsections, we will motivate this setup at more length. Additionally, a more detailed discussion of the architectural details and training of these models can be found in the appendix.

Features as a Decomposition

There is significant empirical evidence suggesting that neural networks have interpretable linear directions in activation space. This includes classic work by Mikolov *et al.* [\[2\]](#) investigating "vector arithmetic" in word embeddings (*but see* [\[3\]](#)) and similar results in other latent spaces (e.g. [\[4\]](#)). There is also a large body of work investigating interpretable neurons, which are just basis dimensions (e.g. *in RNNs* [\[5\]](#); *in CNNs* [\[6\]](#); *in GANs* [\[7\]](#); *but see* [\[8\]](#)). A longer review and discussion of this work can be found in the [Motivation section](#) of *Toy Models* [\[9\]](#), although it doesn't cover more recent work (e.g. [\[10\]](#)).²

If linear directions are interpretable, it's natural to think there's some "basic set" of meaningful directions which more complex directions can be created from. We call these directions *features*, and they're what we'd like to decompose models into. Sometimes, by happy circumstances, individual neurons appear to be these basic interpretable units (see examples above). But quite often, this isn't the case.

Instead, we decompose the activation vector $\mathbf{x}^j$ as a combination of more general features which can be any direction:

$$\mathbf{x}^j \approx \mathbf{b} + \sum_i f_i(\mathbf{x}^j) \mathbf{d}_i \tag{1}$$

where $\mathbf{x}^j$ is the activation vector of length d_{MLP} for datapoint j , $f_i(\mathbf{x}^j)$ is the *activation* of feature i , each $\mathbf{d}_i$ is a unit vector in activation space we call the *direction* of feature i , and $\mathbf{b}$ is a bias.³ Note that this decomposition is not new: it is just a linear matrix factorization of the kind commonly employed in dictionary learning.

In our sparse autoencoder setup, the feature activations are the output of the encoder

$$f_i(x) = \text{ReLU}(W_e(\mathbf{x} - \mathbf{b}_d) + \mathbf{b}_e)_i,$$

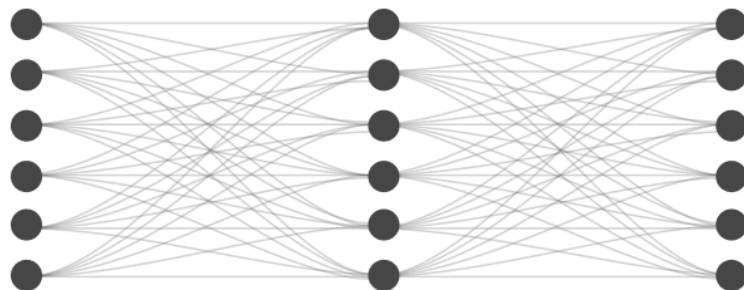
where W_e is the weight matrix of the encoder and $\mathbf{b}_d, \mathbf{b}_e$ are a pre-encoder and an encoder bias. The feature directions are the columns of the decoder weight matrix W_d . (See [appendix](#) for full notation.)

If such a sparse decomposition exists, it raises an important question: are models in some fundamental sense composed of features or are features just a convenient post-hoc description? In this paper, we take an agnostic position, though our [results](#) on feature universality suggest that features have some existence beyond individual models.

SUPERPOSITION HYPOTHESIS

To see how this decomposition relates to superposition, recall that the superposition hypothesis postulates that neural networks “want to represent more features than they have neurons”. We think this happens via a kind of “noisy simulation”, where small neural networks exploit feature sparsity and properties of high-dimensional spaces to approximately simulate much larger much sparser neural networks [1].

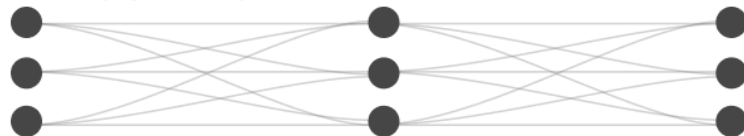
HYPOTHETICAL DISENTANGLED MODEL



Under the superposition hypothesis, the neural networks we observe are **simulations of larger networks** where every neuron is a disentangled feature.



OBSERVED MODEL



These idealized neurons are **projected** on to the actual network as “almost orthogonal” vectors over the neurons.

The network we observe is a **low-dimensional projection** of the larger network. From the perspective of individual neurons, this presents as polysemanticity.

A consequence of this is that we should expect the feature directions to form an *overcomplete basis*. That is, our decomposition should have more directions $\mathbf{d}_i$ than neurons. Moreover, the feature activations should be *sparse*, because sparsity is what enables this kind of noisy simulation. This is mathematically identical to the classic problem of dictionary learning.

What makes a good decomposition?

Suppose that a dictionary exists such that the MLP activation of each datapoint is in fact well approximated by a sparse weighted sum of features as in equation 1. That decomposition will be useful for interpreting the neural network if:

1. We can interpret the conditions under which each feature is active. That is, we have a description of which datapoints j cause the feature to activate (i.e. for which $f_i(\mathbf{x}^j)$ is large) that makes sense on a diverse dataset (including potentially synthetic or off-distribution examples meant to test the hypothesis).⁴
2. We can interpret the downstream effects of each feature, i.e. the effect of changing the value of f_i on subsequent layers. This should be consistent with the interpretation in (1).
3. The features explain a significant portion of the functionality of the MLP layer (as measured by the loss; see How much of the model does our interpretation explain?).

A feature decomposition satisfying these criteria would allow us to:

1. Determine the contribution of a feature to the layer’s output, and the next layer’s activations, on a specific example.

2. Monitor the network for the activation of a specific feature (see e.g. speculation about safety-relevant features).
3. Change the behavior of the network in predictable ways by changing the values of some features. In multilayer models this could look like predictably influencing one layer by changing the feature activations in an earlier layer.
4. Demonstrate that the network has learned certain properties of the data.
5. Demonstrate that the network is using a given property of the data in producing its output on a specific example.⁵
6. Design inputs meant to activate a given feature and elicit certain outputs.

Of course, decomposing models into components is just the beginning of the work of mechanistic interpretability! It provides a foothold on the inner workings of models, allowing us to start in earnest on the task of unraveling circuits and building a larger-scale understanding of models.

Why not use architectural approaches?

In [Toy Models of Superposition](#) [\[1\]](#), we highlighted several different approaches to solving superposition. One of those approaches was to engineer models to simply not have superposition in the first place.

Initially, we thought that this might be possible but come with a large performance hit (i.e. produce models with greater loss). Even if this performance hit had been too large to use in practice for real models, we felt that success at creating monosemantic models would have been very useful for research, and in a lot of ways this felt like the "cleanest" approach for downstream analysis.

Unfortunately, having spent a significant amount of time investigating this approach, we have ultimately concluded that it is more fundamentally non-viable.

In particular, we made several attempts to induce activation sparsity during training to produce models without superposition, even to the point of training models with 1-hot activations. This indeed eliminates superposition, but it fails to result in cleanly-interpretable neurons! Specifically, we found that individual neurons can be polysemantic even in the absence of superposition. This is because in many cases models achieve lower loss by representing multiple features ambiguously (in a polysemantic neuron) than by representing a single feature unambiguously and ignoring the others.

To understand this, consider a toy model with a single neuron trained on a dataset with four mutually-exclusive features (A/B/C/D), each of which makes a distinct (correct) prediction for the next token, labeled in the same fashion. Further suppose that this neuron's output is binary: it either fires or it doesn't. When it fires, it produces an output vector representing the probabilities of the different possible next tokens.

We can calculate the cross-entropy loss achieved by this model in a few cases:

1. Suppose the neuron only fires on feature A, and correctly predicts token A when it does. The model ignores all of the other features, predicting a uniform distribution over tokens B/C/D when feature A is not present. In this case the loss is $\frac{3}{4} \ln 3 \approx 0.8$.
2. Instead suppose that the neuron fires on both features A and B, predicting a uniform distribution over the A and B tokens. When the A and B features are not present, the model predicts a uniform distribution over the C and D tokens. In this case the loss is $\ln 2 \approx 0.7$.

Because the loss is lower in case (2) than in case (1), the model achieves better performance by making its sole neuron polysemantic, even though there is no superposition.

This example might initially seem uninteresting because it only involves one neuron, but it actually points at a general issue with highly sparse networks. If we push activation sparsity to its limit, only a single neuron will activate at a time. We can now consider that single neuron and the cases where it fires. As seen earlier, it can still be advantageous for that neuron to be polysemantic.

Based on this reasoning, and the results of our experiments, we believe that models trained on cross-entropy loss will generally prefer to represent more features polysemantically than to represent fewer "true features" monosemantically, even in cases where sparsity constraints make superposition impossible.

Models trained on other loss functions do not necessarily suffer this problem. For instance, models trained under mean squared error loss (MSE) may achieve the same loss for both polysemantic and monosemantic representations (e.g. [1]), and for some feature importance curves they may actively prefer the monosemantic solution [2]. But language models are not trained with MSE, and so we do not believe architectural changes can be used to create fully monosemantic language models.

Note, however, that in learning to decompose models post-training we do use an MSE loss (between the activations and their representation in terms of the dictionary), so sparsity can inhibit superposition from forming in the learned dictionary. (Otherwise, we might have superposition "all the way down.")

Using Sparse Autoencoders To Find Good Decompositions

There is a long-standing hypothesis that many natural latent variables in the world are sparse (see [3], section "Why Sparseness?"). Although we have a limited understanding of the features that exist in language models, the examples we do have (e.g. [4]) are suggestive of highly sparse variables. Our work on Toy Models of Superposition [5] shows that a large set of sparse features could be represented in terms of directions in a lower dimensional space.

For this reason, we seek a decomposition which is *sparse* and *overcomplete*. This is essentially the problem of sparse dictionary learning [6]. (Note that, since we're only asking for a sparse decomposition of the activations, we make no use of the downstream effects of $\mathbf{d}_i$ on the model's output. This means we'll be able to use those downstream effects in later sections as a kind of validation on the features found.)

It's important to understand why making the problem *overcomplete* – which might initially sound like a trivial change – actually makes this setting very different from similar approaches seeking sparse disentanglement in the literature. It's closely connected to why dictionary learning is such a non-trivial operation; in fact, as we'll see, it's actually kind of miraculous that this is possible at all. At the heart of dictionary learning is an *inner problem* of computing the feature activations $f_i(\mathbf{x})$ for each datapoint $\mathbf{x}$, given the feature directions $\mathbf{d}_i$. On its surface, this problem may seem impossible: we're asking to determine a high-dimensional vector from a low-dimensional projection. Put another way, we're trying to invert a very rectangular matrix. The only thing which makes it possible is that we are looking for a high-dimensional vector that is sparse! This is the famous and well-studied problem of compressed sensing, which is NP-hard in its exact form. It is possible to store high-dimensional sparse structure in lower-dimensional spaces, but recovering it is hard.

Despite its difficulty, there are a host of sophisticated methods for dictionary learning (e.g. [1]). These methods typically involve optimizing a relaxed problem or doing a greedy search. We tried several of these classic methods, but ultimately decided to focus on a simpler sparse autoencoder approximation of dictionary learning (similar to Sharkey, *et al.* [2]). This was for two reasons. First, a sparse autoencoder can readily scale to very large datasets, which we believe is necessary to characterize the features present in a model trained on a large and diverse corpus.⁶ Secondly, we have a concern that iterative dictionary learning methods might be "too strong", in the sense of being able to recover features from the activations which the model itself cannot access. Exact compressed sensing is NP-hard, which the neural network certainly isn't doing. By contrast, a sparse autoencoder is very similar in architecture to the MLP layers in language models, and so should be similarly powerful in its ability to recover features from superposition.

Sparse Autoencoder Setup

We briefly overview the architecture and training of our sparse autoencoder here, and provide further details in [Basic Autoencoder Training](#). Our sparse autoencoder is a model with a bias at the input, a linear layer with bias and ReLU for the encoder, and then another linear layer and bias for the decoder. In toy models we found that the bias terms were quite important to the autoencoder's performance.⁷

We train this autoencoder using the Adam optimizer to reconstruct the MLP activations of our transformer model, with an MSE⁸ loss plus an L1 penalty to encourage sparsity.

In training the autoencoder, we found a couple of principles to be quite important. First, scale really matters. We found that training the autoencoder on more data made features subjectively "sharper" and more interpretable. In the end, we decided to use 8 billion training points for the autoencoder (see [Autoencoder Dataset](#)).

Second, we found that over the course of training some neurons cease to activate, even across a large number of datapoints. We found that "resampling" these dead neurons during training gave better results by allowing the model to represent more features for a given autoencoder hidden layer dimension. Our resampling procedure is detailed in [Neuron Resampling](#), but in brief we periodically check for neurons which have not fired in a significant number of steps and reset the encoder weights on the dead neurons to match data points that the autoencoder does not currently represent well.

For readers looking to apply this approach, we supply an appendix with [Advice for Training Sparse Autoencoders](#).

HOW CAN WE TELL IF THE AUTOENCODER IS WORKING?

Usually in machine learning we can quite easily tell if a method is working by looking at an easily-measured quantity like the test loss. We spent quite some time searching for an equivalent metric to guide our efforts here, and unfortunately have yet to find anything satisfactory.

We began by looking for [an information-based metric](#), so that we could say in some sense that the best factorization is the one that minimizes the total information of the autoencoder and the data. Unfortunately, this total information did not generally correlate with subjective feature interpretability or activation sparsity. (Runs whose feature activations had an average L0 norm in the hundreds but low reconstruction error could have lower total information than those with smaller average L0 norm and higher reconstruction error.)

Thus we ended up using a combination of several additional metrics to guide our investigations:

1. **Manual inspection:** Do the features seem interpretable?

2. **Feature density:** we found that the number of “live” features and the percentage of tokens on which they fire to be an extremely useful guide. (See appendix for [details](#).)
3. **Reconstruction loss:** How well does the autoencoder reconstruct the MLP activations? Our goal is ultimately to explain the function of the MLP layer, so the MSE loss should be low.
4. **Toy models:** Having toy models where we know the ground truth and so can cleanly evaluate the autoencoder’s performance was crucial to our early progress.

Interpreting or measuring some of these signals can be difficult, though. For instance, at various points we thought we saw features which at first didn’t make any sense, but with deeper inspection we could understand. Likewise, while we have identified some desiderata for the distribution of feature densities, there is much that we still do not understand and which prevents this from providing a clear signal of progress.

We think it would be very helpful if we could identify better metrics for dictionary learning solutions from sparse autoencoders trained on transformers.

The (one-layer) model we're studying

We chose to study a one-layer transformer model. We view this model as a testbed for dictionary learning, and in that role it brings three key advantages:

1. Because one-layer models are small they likely have fewer “true features” than larger models, meaning features learned by smaller dictionaries might cover all their “true features”. Smaller dictionaries are cheaper to train, allowing for fast hyperparameter optimization and experimentation.
2. We can highly overtrain a one-layer transformer quite cheaply. We hypothesize that a very high number of training tokens may allow our model to learn cleaner representations in superposition.
3. We can easily analyze the effects features have on the logit outputs, because these are approximately linear in the feature activations.⁹ This provides helpful corroboration that the features we have found are not just telling us about the data distribution, and actually reflect the functionality of the model.

We trained two one-layer transformers with the same hyperparameters and datasets, differing only in the random seed used for initialization. We then learned dictionaries of many different sizes on both transformers, using the same hyperparameters for each matched pair of dictionaries but training on the activations of different tokens for each transformer.

We refer to the main transformer we study in this paper as the “A” transformer. We primarily use the other transformer (“B”) to study feature universality, as we can e.g. compare features learned from the “A” and “B” transformers and see how similar they are.

Notation for Features

Throughout this draft, we’ll use strings like “A/1/2357” to denote features. The first portion “A” or “B” denote which model the features come from. The second part (e.g. the “1” in “A/1”) denotes the dictionary learning run. These vary in the number of learned factors and the L1 coefficient used. A table of all of our runs is available [here](#). Notably, A/0...

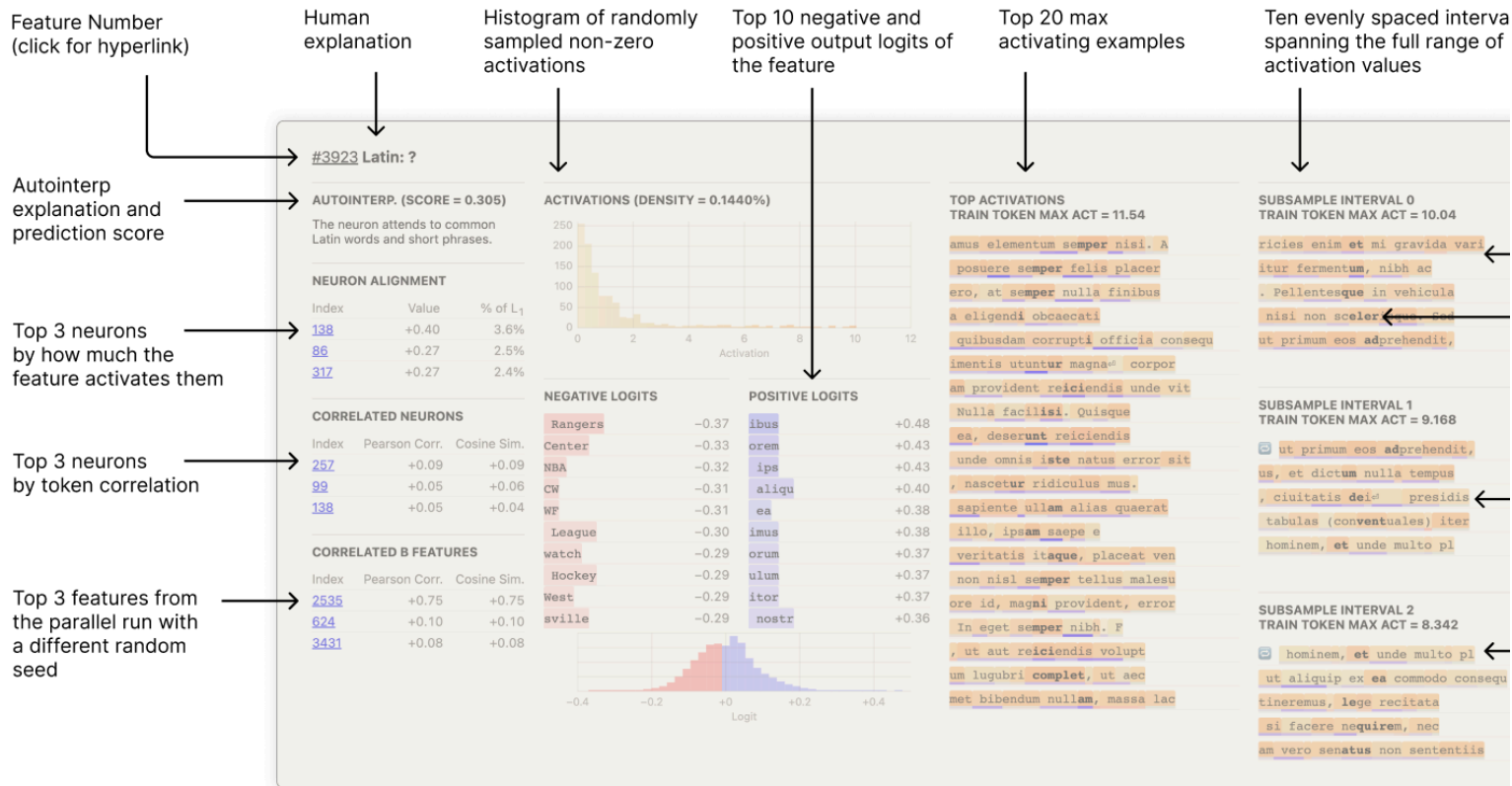
A/5 form a sequence with fixed L1 coefficients and increasing dictionary sizes. The final portion (e.g. the "2357" in "A/1/2357") corresponds to the specific feature in the run.

Sometimes, we want to denote neurons from the transformer rather than features learned by the sparse autoencoder. In this case, we use the notation "A/neurons/32".

Interface for Exploring Features

We provide an interface for exploring all the features in all our dictionary learning runs. Links to the visualizations for each run can be found [here](#). We suggest beginning with the [interface for A/1](#), which we discuss the most.

These interfaces provide extensive information on each feature. This includes examples of when they activate, what effect they have on the logits when they do, examples of how they affect the probability of tokens if the feature is ablated, and much more:



Our interface also allows users to search through features:

Search among the features

Where the search query is matched

Which color the

Search

in

Sort by

Un

Go

Case sensitive

Top examples

Consistent Activation Heuristic

F

✓ Case sensitive

Case insensitive

Regex (case sensitive)

Regex (case insensitive)

Exact (not substring)

✓ Top examples

Top examples (individual tokens)

Logits

Top examples and logits

Top example tokens and logits

Index and name

Autointerpretation

✓ Consistent Activation Heuristic

Max Activation

Min Activation

Max Dense

Min Dense

Max Neuron Aligned

Min Neuron Aligned

Max Activation / Density

Index

Random

Search Relevance

Autointerpretation Score

✓

"f

lc

fe

"f

th

o'

re

a

Additionally, we provide a [second interface](#) displaying all features active on a given dataset example. This is available for a set of example texts.

For a selection of texts, we show all the active features.

Hovering over a token will reveal the top activating features for that token

Hovering over a feature will reveal the top activating tokens on each of

<EOT>O Attic shape! Fair attitude! with brede
Of marble men and maidens overwrought,
With forest branches and the trodden weed;
Thou, silent form, dost tease us out of thought
As doth eternity: Cold Pastoral!
When old age shall this generation waste,
Thou shalt remain, in midst of other woe
Than ours, a friend to man, to whom thou say'st,
"Beauty is truth, truth beauty,—that is all
Ye know on earth, and all ye need to know."

Index	Act	% of Max	Autointerpreted Label
#447	1.281	19.03%	This neuron fires when it sees words commonly associated with elevated/formal language and old-fashioned styles, such as older literature. It attends to words like "thou", "hast", "thy", "verily", archaic verb forms ("knowest", "fathered"), and titles like "lord" and "sir".
#448	0.451	7.60%	This neuron seems to attend to formal or archaic-sounding religious language, with a focus on worshipping, professing belief, preaching, praying, invoking God, etc.
#227	0.291	3.33%	This neuron appears to attend to various types of medical and biomedical terminology. It activates on words and phrases related to medical conditions, treatments, procedures, diagnoses, anatomy, and other biomedical concepts.
#504	0.211	2.42%	This neuron seems to fire when there is a comma followed by the word "and".

<EOT>O Attic shape! Fair attitude! with brede
Of marble men and maidens overwrought,
With forest branches and the trodden weed;
Thou, silent form, dost tease us out of thought
As doth eternity: Cold Pastoral!
When old age shall this generation waste,
Thou shalt remain, in midst of other woe
Than ours, a friend to man, to whom thou say'st,
"Beauty is truth, truth beauty,—that is all
Ye know on earth, and all ye need to know."

Index	Act	% of Max	Autointerpreted Label
#447	3.026	44.95%	This neuron fires when it sees words commonly associated with elevated/formal language and old-fashioned styles, such as older literature. It attends to words like "thou", "hast", "thy", "verily", archaic verb forms ("knowest", "fathered"), and titles like "lord" and "sir".
#174	0.825	13.75%	The neuron fires when it sees the term "here" and words related to talking about facts or sharing information.
#227	0.485	5.55%	This neuron appears to attend to various types of medical and biomedical terminology. It activates on words and phrases related to medical conditions, treatments, procedures, diagnoses, anatomy, and other biomedical concepts.
#496	0.340	5.20%	This neuron fires when it sees present participle verb forms (-ing) as well as some similar sounding endings like -ington, -ation, etc.

<EOT>O Attic shape! Fair attitude! with brede
Of marble men and maidens overwrought,
With forest branches and the trodden weed;
Thou, silent form, dost tease us out of thought
As doth eternity: Cold Pastoral!
When old age shall this generation waste,
Thou shalt remain, in midst of other woe
Than ours, a friend to man, to whom thou say'st,
"Beauty is truth, truth beauty,—that is all
Ye know on earth, and all ye need to know."

Index	Act	% of Max	Autointerpreted Label
#447	1.281	19.03%	This neuron fires when it sees words commonly associated with elevated/formal language and old-fashioned styles, such as older literature. It attends to words like "thou", "hast", "thy", "verily", archaic verb forms ("knowest", "fathered"), and titles like "lord" and "sir".
#448	0.451	7.60%	This neuron seems to attend to formal or archaic-sounding religious language, with a focus on worshipping, professing belief, preaching, praying, invoking God, etc.
#227	0.291	3.33%	This neuron appears to attend to various types of medical and biomedical terminology. It activates on words and phrases related to medical conditions, treatments, procedures, diagnoses, anatomy, and other biomedical concepts.
#504	0.211	2.42%	This neuron seems to fire when there is a comma followed by the word "and".

Detailed Investigations of Individual Features

The most important claim of our paper is that dictionary learning can extract features that are significantly more monosemantic than neurons. In this section, we give a detailed demonstration of this claim for a small number of features which activate in highly specific contexts.

The features we study respond to

- Text written in Arabic script (like "الكتاب المختصر في حساب الجبر والمقابلة")
- DNA sequences (like "CCTGGTACTGTACGAACGAACGACGTAGCCTTGG")
- base64 strings (like the final characters in "https://www.youtube.com/watch?v=dQw4w9WgXcQ")
- Text written in Hebrew script (like "בראשית ברא אלהים את השמים ואת הארץ")

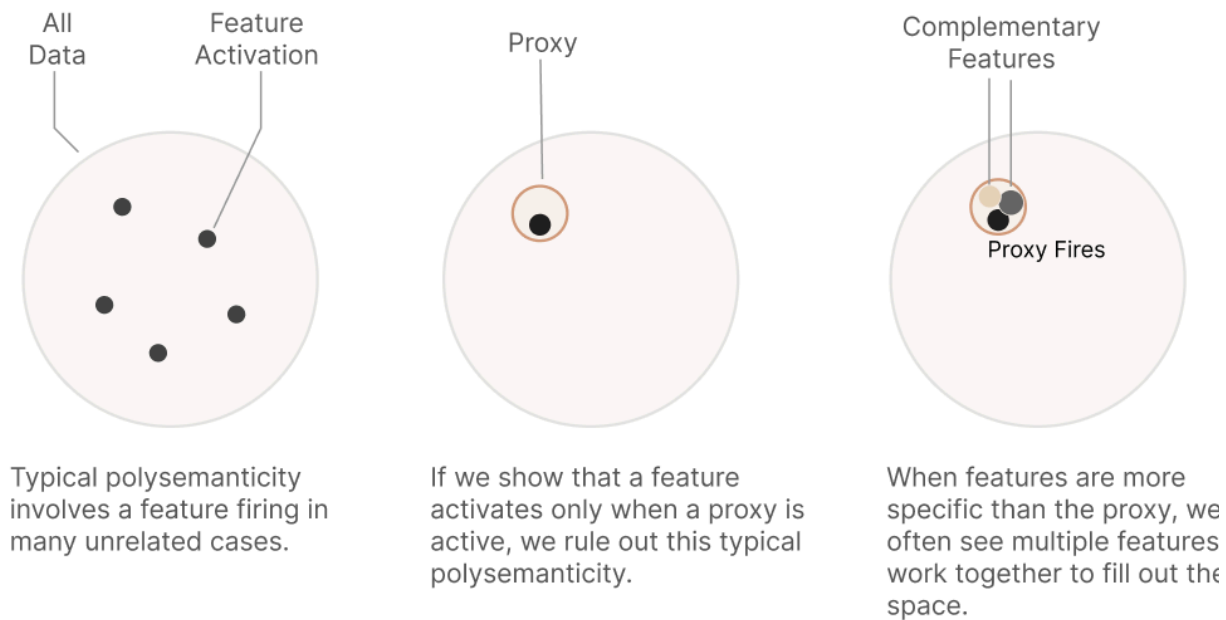
For each learned feature, we attempt to establish the following claims:

1. The learned feature *activates with high specificity* for the hypothesized context. (When the feature is on the context is usually present.)
2. The learned feature *activates with high sensitivity* for the hypothesized context. (When the context is present, the feature is usually on.)
3. The learned feature *causes* appropriate downstream behavior.
4. The learned feature *does not correspond to any neuron*.
5. The learned feature is *universal* – a similar feature is found by dictionary learning applied to a different model.

To demonstrate claims 1–3, we devise computational proxies for each context, numerical scores estimating the (log-)likelihood that a string (or token) is from the specific context. The contexts chosen above are easy to model based on the defined sets of unicode characters involved. We model DNA sequences as random strings of characters from [ATCG] and we model base64 strings as random sequences of characters from [a-zA-Z0-9+/]. For Arabic script and Hebrew features, we exploit the fact that each language is written in a script consisting of well-defined Unicode blocks. Each computational proxy is then an estimate of the log-likelihood ratio of a string under the hypothesis versus under the full empirical distribution of the dataset. The full description of how we estimate $\log(P(s|\text{context})/P(s))$ for each feature hypothesis is given in the [appendix section on proxies](#).

In this section we primarily study the learned feature which is most active in each context. There are typically other features that also model that context, and we find that rare “gaps” in the sensitivity of a main feature are often explained by the activity of another. We discuss this phenomenon in detail in sections on [Activation Sensitivity](#) and [Feature Splitting](#).

We take pains to demonstrate the specificity of each feature, as we believe that to be more important for ruling out polysemanticity. Polysemanticity typically involves neurons activating for clearly unrelated concepts.¹⁰ If our proxy shows that a feature only activates in some relatively rare and specific context, that would exclude the typical form of polysemanticity.



We finally note that the features in this section are cherry-picked to be *easier to analyze*. Defining simple computational proxies for most features we find, such as text concerning fantasy games, would be difficult, and we analyze them in other ways in the following section.

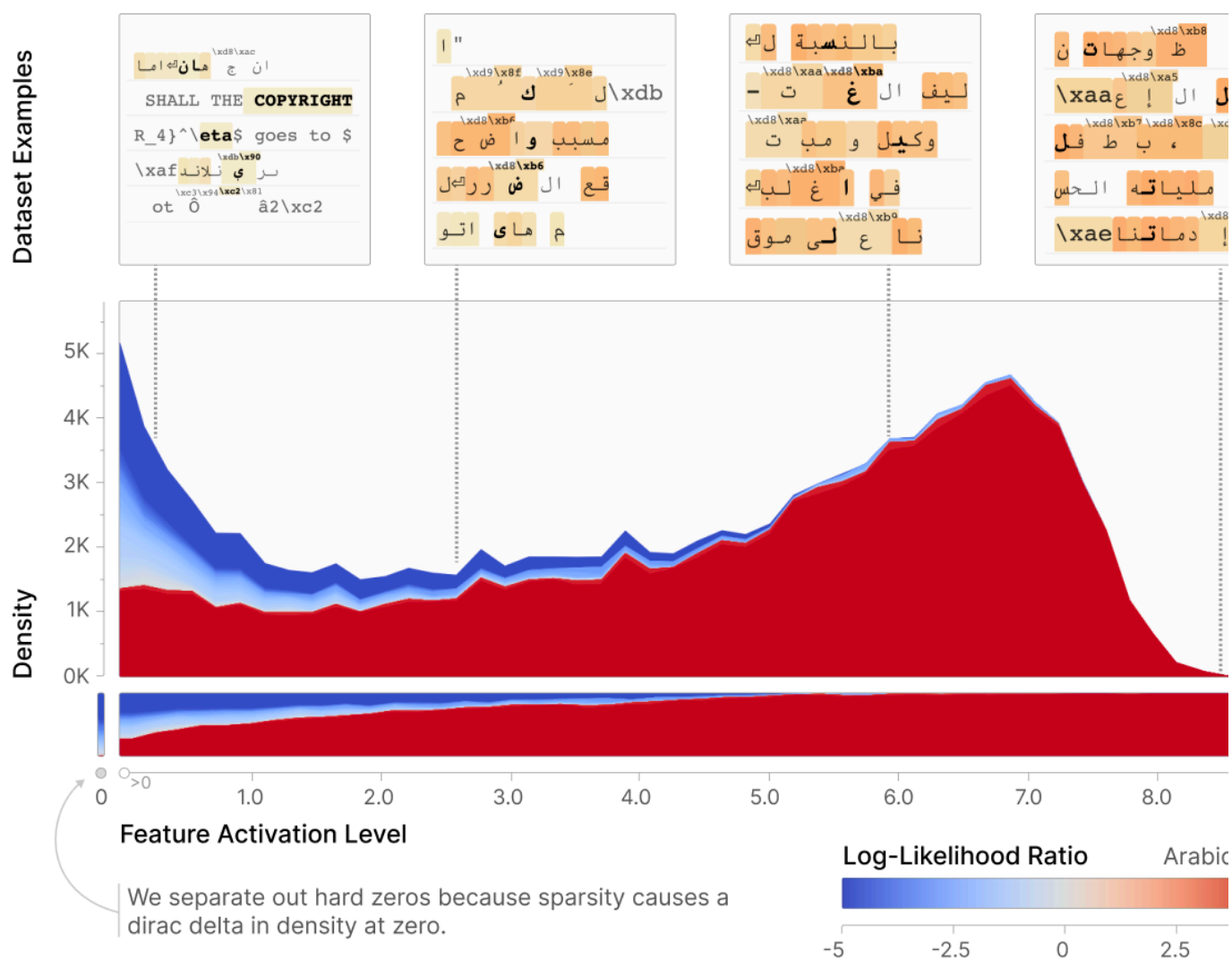
Arabic Script Feature

The first feature we'll consider is an Arabic Script feature, A/1/3450. It activates in response to text in Arabic, Farsi, Urdu (and possibly other languages), which use the Arabic script. This feature is quite specific and relatively sensitive to Arabic script, and effectively invisible if we view the model in terms of individual neurons.

ACTIVATION SPECIFICITY

Our first step is to show that this feature fires almost exclusively on text in Arabic script. We give each token an "Arabic script" score using an estimated likelihood ratio $\log(P(s|\text{Arabic Script})/P(s))$, and break down the histogram of feature activations by that score. Arabic text is quite rare in our overall data distribution – just 0.13% of training tokens – but it makes up 81% of the tokens on which our feature is active. That percentage varies significantly by feature activity level, from 25% when the feature is barely active to 98% when the feature activation is above 5.

Feature Activation Distribution (A/1/3450)

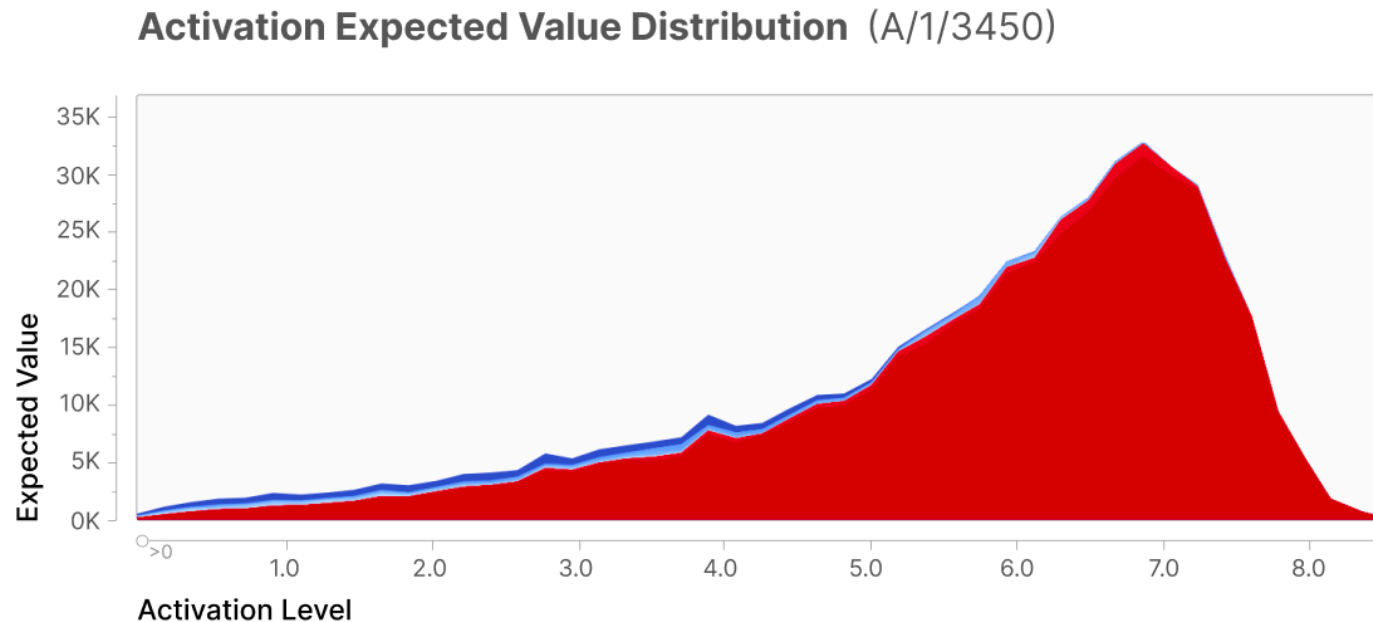


We also show dataset examples demonstrating different levels of feature activity. In interpreting them, it's important to note that Arabic Unicode characters are often split into multiple tokens. For example, the character `ث` (U+062B) is tokenized as `\xd8` followed by `\xab`.¹¹ When this happens, A/1/3450 typically only fires on the *last* token of the Unicode character.

The upper parts of the activation spectrum, above an activity of ~5, clearly respond with high specificity to Arabic script. What should we make of the lower portions? We have three hypotheses:

- **The proxy is imperfect.** It has false negatives due to common characters which are in other Unicode blocks (e.g., whitespace, punctuation), and also due to places where tokenization has created strange behavior.¹²
- **The model may be imperfect (but still calibrated).** If features activate proportional to their "confidence" that some property is present, then we should expect them to sometimes be wrong in their weak activations. Although it might seem silly to fail to recognize which language a piece of text is, this is a very weak one layer transformer model, and the fact that characters are often split into multiple tokens adds additional difficulty.
- **The autoencoder may be imperfect:** If the width of our autoencoder is less than the number of "true features" being used by the model, then unrecovered features may show up as low activations across many of our learned features.

Regardless, large feature activations have larger impacts on model predictions,¹³ so getting their interpretation right matters most. One useful tool for assessing the impact of false positives at low activation levels is the "expected value plot" of Cammarata *et al.* [14]. We plot the distribution of feature activations *weighted by activation level*. Most of the magnitude of activation provided by this feature comes from dataset examples which are in Arabic script.



ACTIVATION SENSITIVITY

In the Feature Activation Distribution above, it's clear that A/1/3450 is not sensitive to all tokens in Arabic script. In the random dataset examples, it fails to fire on five examples of the prefix "ال", transliterated as "al-", which is the equivalent of the definite article "the" in English. However, in exactly those places, another feature which is specific to Arabic script, A/1/3134, fires. There are several additional features that fire on Arabic and related scripts (e.g. A/1/1466, A/1/3134, A/1/3399) which contribute to representing Arabic script. Another example deals with Unicode tokenization: when Arabic characters are split into multiple tokens, the feature we analyze here only activates at the final token comprising the character, while A/1/3399 activates on the first token comprising the character. To see how these features collaborate, we provide an [alternative visualization](#) showing all the features active on a snippet of Arabic text. We consider such interactions more in the [Phenomenology](#) section below.

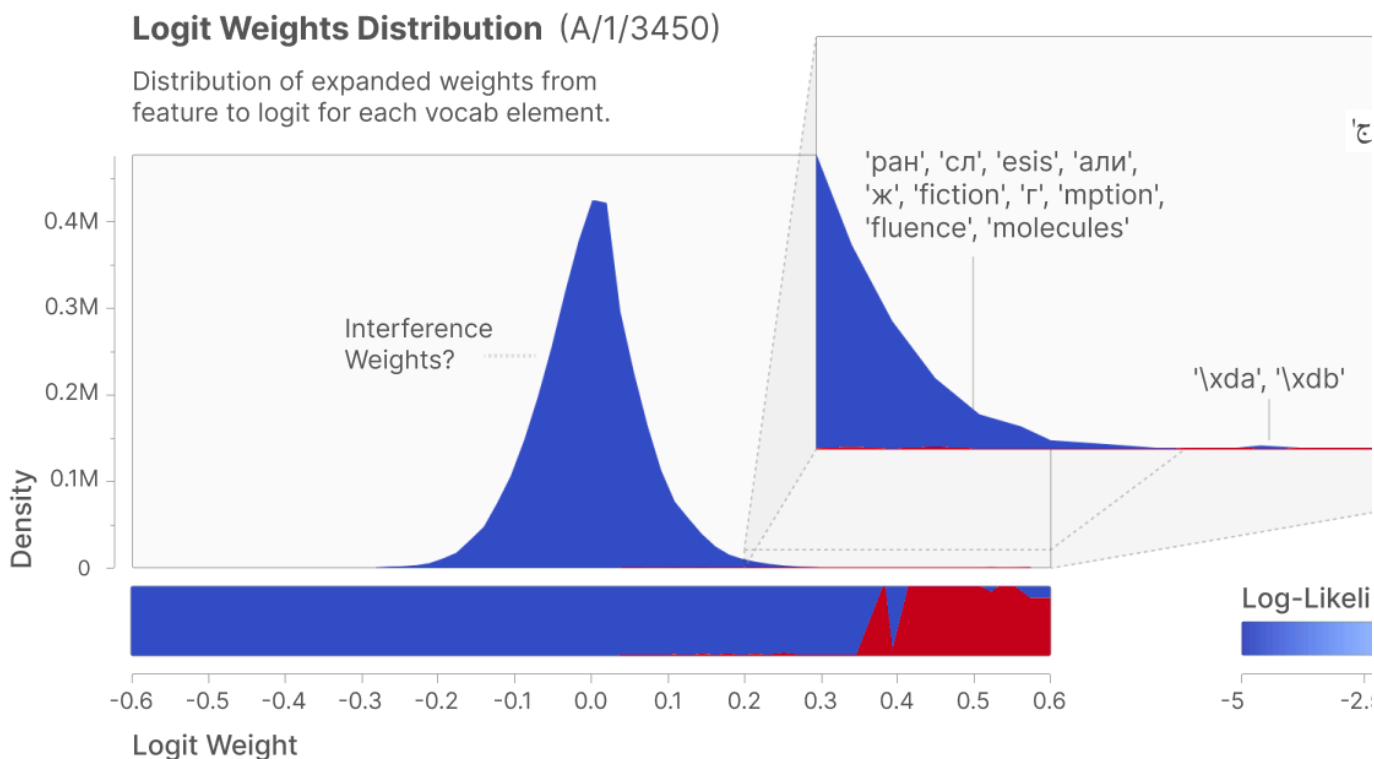
Nevertheless, we find a Pearson correlation of 0.74 between the activity of our feature and the activity of the Arabic script proxy (thresholded at 0), over a dataset of 40 million tokens. Correlation provides a joint measure of sensitivity and specificity that takes magnitude into account, and 0.74 is a substantial correlation.

FEATURE DOWNSTREAM EFFECTS

Because the autoencoder is trained on model activations, the features it learns could in theory represent structure in the training data alone, without any relevance to the network's function. We show instead that the learned features have interpretable causal effects on model outputs which make sense in light of the features' activations. Note that these downstream effects are not inputs to the dictionary learning process, which only sees the activations of the MLP layer. If the resulting features also mediate important downstream *behavioral effects* then we can be confident that the feature is truly connected to the MLP's functional role in the network and not just a property of the underlying data.

We begin with a linear approximation to the effect of each feature on the model logits. We compute the logit weight following the path expansion approach of [14], multiplying each feature direction by the MLP output weights, an approximation of the layer norm operation combining a projection to remove the mean and a diagonal matrix approximating the scaling terms, and the unembedding matrix ($d_i W_{down} \pi L W_{unembed}$). Because the softmax function is shift-invariant there is no absolute scale for the logit weights; we shift these so that the median logit weight for each feature is zero.

Each feature, when active, makes some output tokens more likely and some output tokens less likely. We plot that distribution of logit weights.¹⁵ There is a large primary mode at zero, and a second much smaller mode on the far right, the tokens whose likelihood most increases when our feature is on. This second mode appears to correspond to Arabic characters, and tokens which help represent Arabic script characters (especially `\xd8` and `\xd9`, which are often the first half of the UTF-8 encodings of Arabic Unicode characters in the basic Arabic Unicode block).

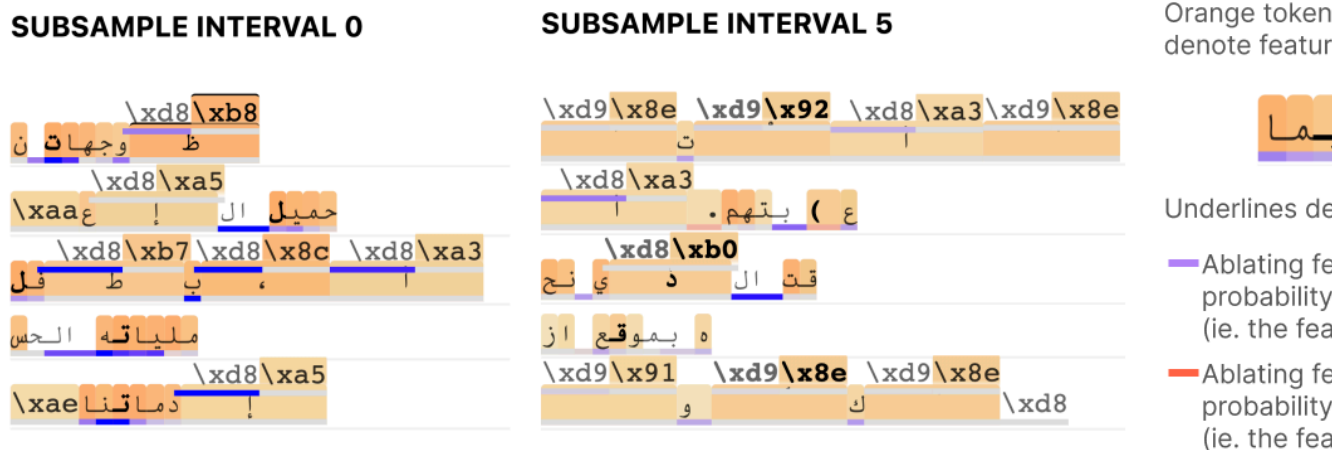


This suggests that activating this feature increases the probability the network predicts Arabic script tokens.¹⁶

To visualize these effects on actual data, we causally ablate the feature. For a given dataset example, we run the context through the model until the MLP layer, decode the activations into features, then subtract off the activation of A/1/3450, artificially setting it to zero on the whole context, before applying the rest of the model. We visualize the effect of ablating the feature using underlines in the visualization; tokens whose predictions were helped by the feature (ablation decreased likelihood) are underlined in blue and tokens whose predictions were hurt by the feature (ablation increased likelihood) are underlined in red.

In the example on the right below we see that the A/1/3450 was active on every token in a short context (orange background). Ablating it hurt the predictions of all the tokens in Arabic script (purple underlines), but helped the prediction of the period . (orange underline). The rest of the figure displays contexts from two different ranges of feature activation levels. (The feature activation on the middle token of examples on the right ("subsample interval 5") is about half that of the middle token of examples on the left ("subsample interval 0")). We see that the feature was causally helping the model predictions on Arabic script through that full range, and the only tokens made less likely by the feature are punctuation shared with other scripts. The magnitudes of the impact are larger when the feature is more active.

Feature Ablations (A/1/3450)



We encourage interested readers to view the feature visualization for A/1 to review this and other effects.

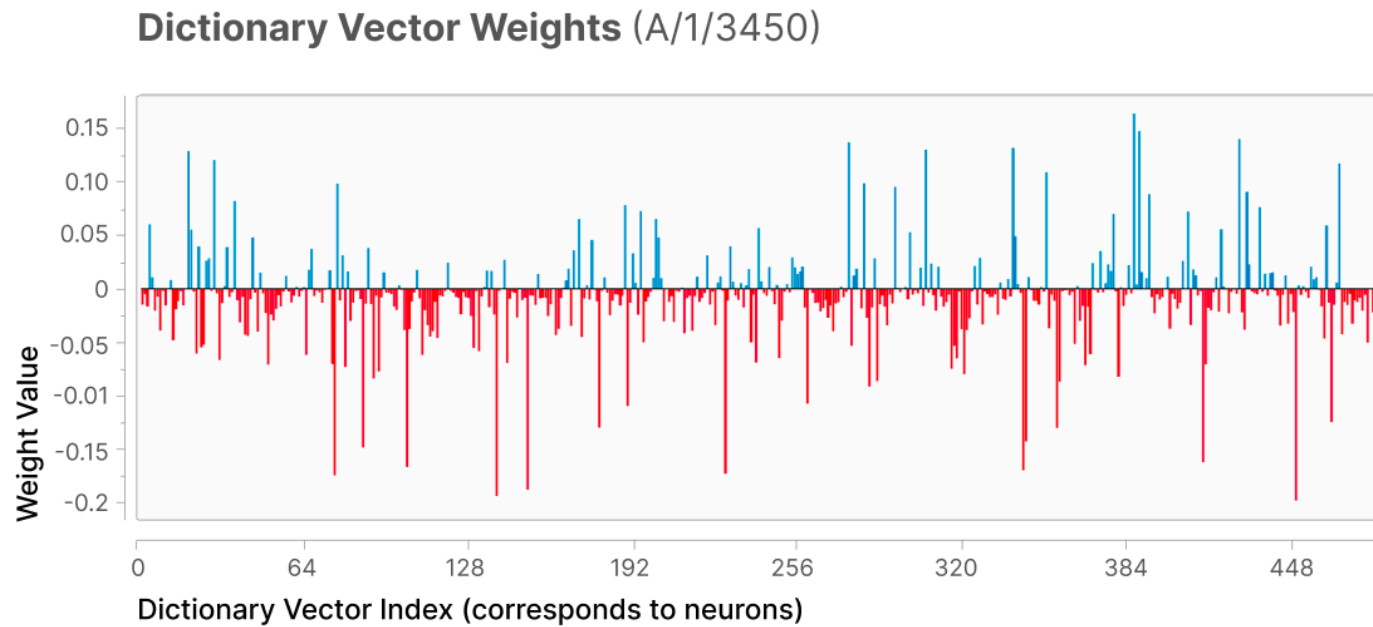
We also validate that the feature's downstream effect is in line with our interpretation as an Arabic script feature by sampling from the model with the feature activity "pinned" at a high value. To do this, we start with a prefix `1,2,3,4,5,6,7,8,9,10` where the model has an expected continuation (keep in mind that this is a one layer model that is very weak!). We then instead set A/1/3450 to its maximum observed value and see how that changes the samples:

Prefix	Normal Samples	Prefix	Pinned Samples
1,2,3,4,5,6,7,8,9,10	,8,3,3,3,3,3 ,8,8,8,9,7,6 ,8,30,20,8,10,10 ,10,10,10,10,10,10,20 ,10,7,6,8,8,6,8	1,2,3,4,5,6,7,8,9,10	يسوإدارية ال أفروكايد يدولة عم تعفات البعدي يان اسم استخ

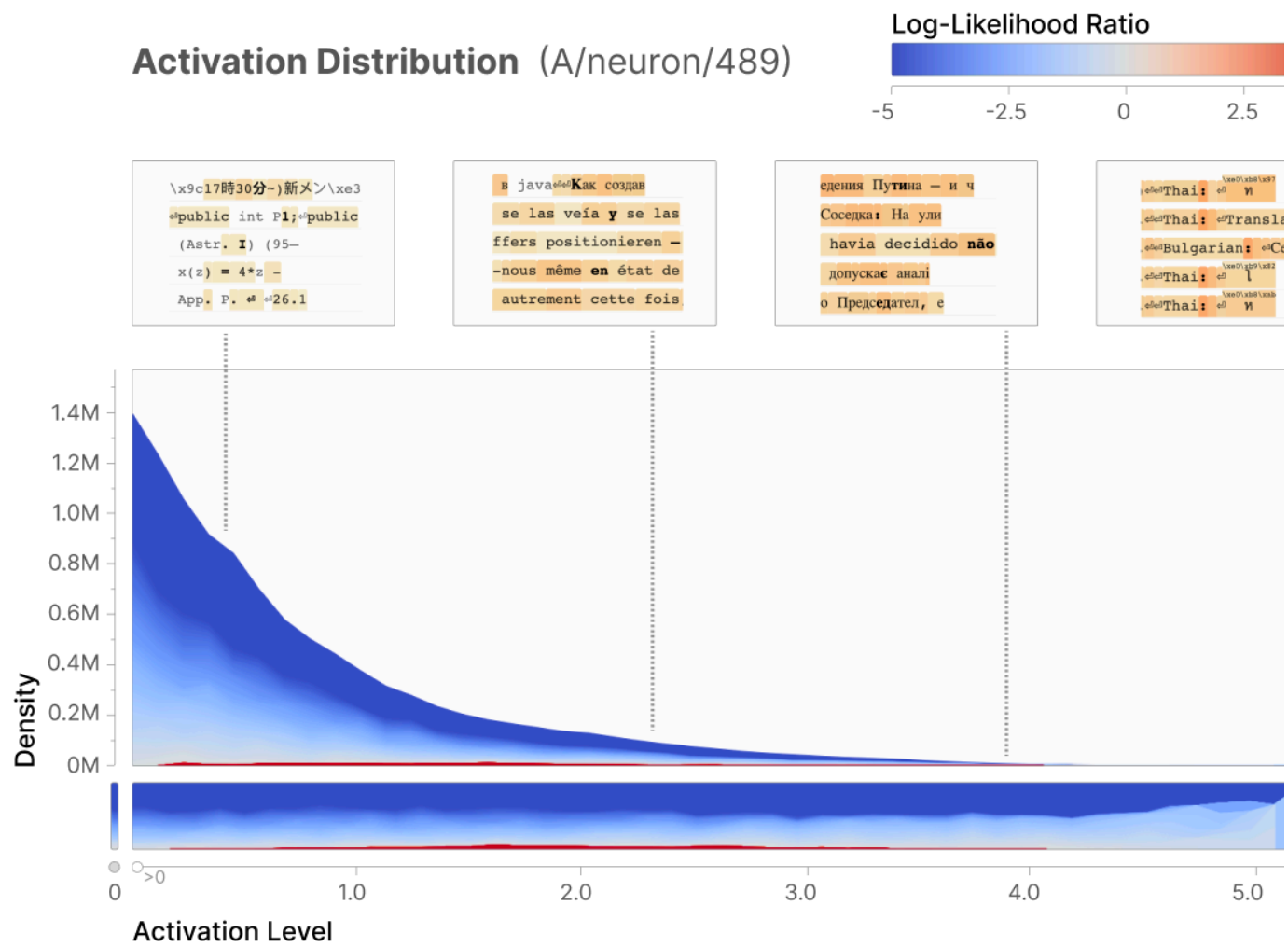
THE FEATURE IS NOT A NEURON

This feature seems rather monosemantic, but some models have relatively monosemantic neurons, and we want to check that dictionary learning didn't merely hand us a particularly nice neuron.¹⁷ We first note that when we search for neurons with Arabic script in their top dataset 20 examples, only one neuron turns up, with a single example in Arabic script. (Eighteen of the remaining examples are in English, and one is in Cyrillic.)

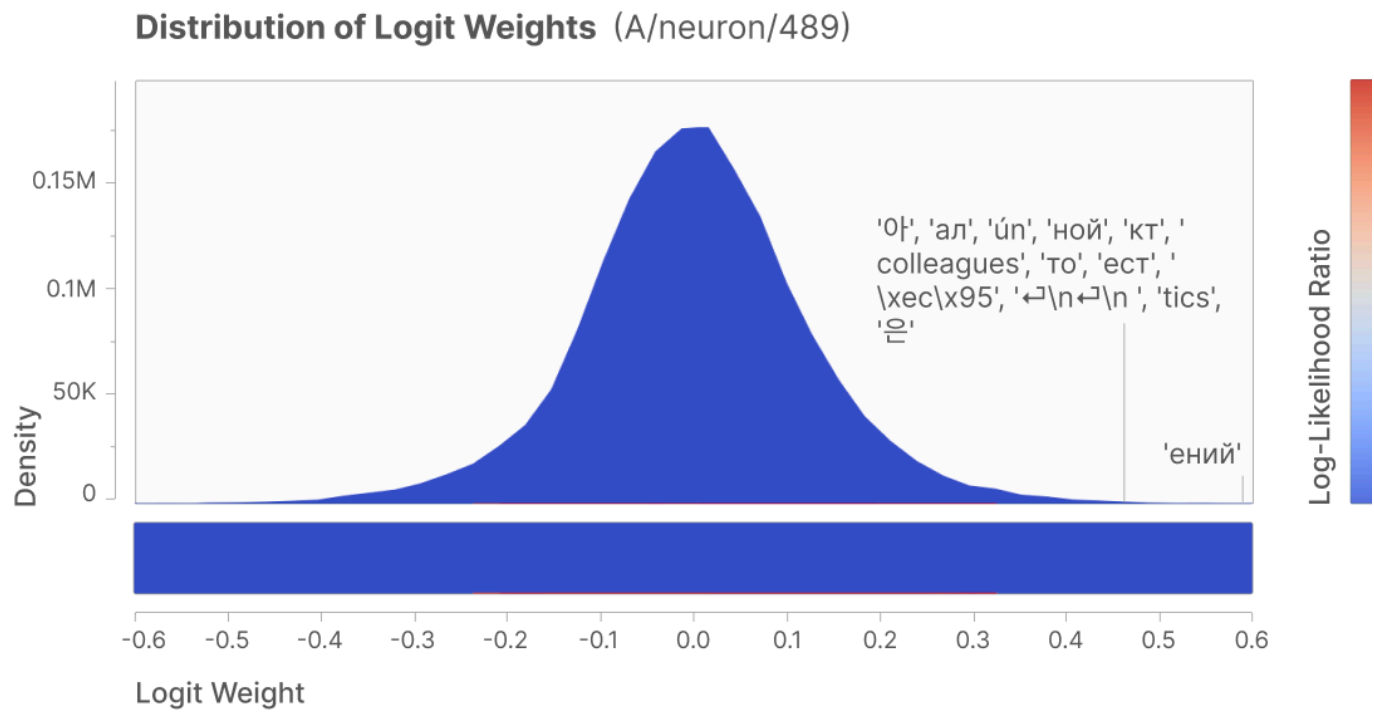
We then look at the coefficients of the feature in the neuron basis, and find that the three largest coefficients by magnitude are all negative (!) and there are a full 27 neurons whose coefficients are at least 0.1 in magnitude.



It is of course possible that these neurons engage in a delicate game of cancellation, resulting in one particular neuron's primary activations being sharpened. To check for this, we find the neuron whose activations are most correlated to the feature's activations over a set of ~40 million dataset examples.¹⁸ The most correlated neuron (A/neurons/489) responds to a mixture of different non-English languages.¹⁹ We can see this by visualizing the activation distribution of the neuron – the dataset examples are all other languages, with Arabic script present only as a small sliver.



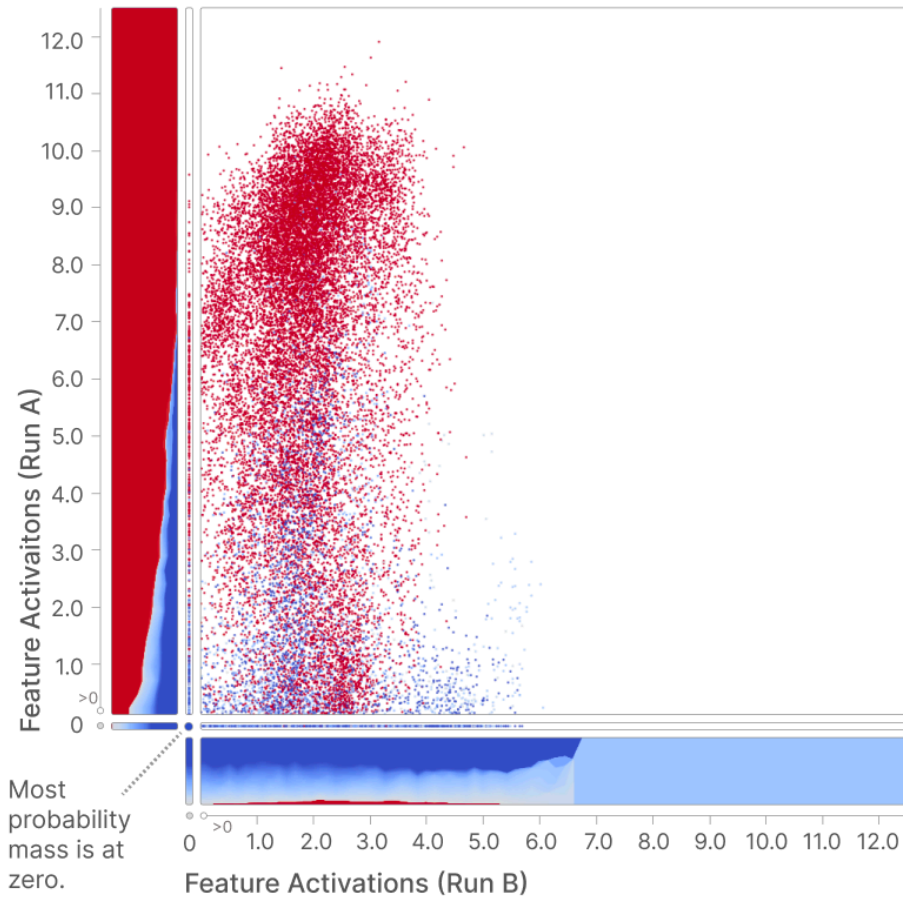
Logit weight analysis is also consistent with this neuron responding to a mixture of languages. For example, in the figure below many of the top logit weights appear to include Russian and Korean tokens. Careful readers will observe a thin red sliver corresponding to rare Arabic script tokens in the distribution. These Arabic script tokens have weight values that are very slightly positive leaning overall, but some are negative.



Finally, scatter plots and correlations suggest the similarities between A/1/3450 and the neuron are non-zero, but quite minimal.²⁰ In particular, note how the y-axis of the logit weights scatter plot (corresponding to the feature) cleanly separates the Arabic script token logits, while the x-axis does not. More generally, note how the y-marginals both clearly exhibit specificity, but the x-marginals do not.

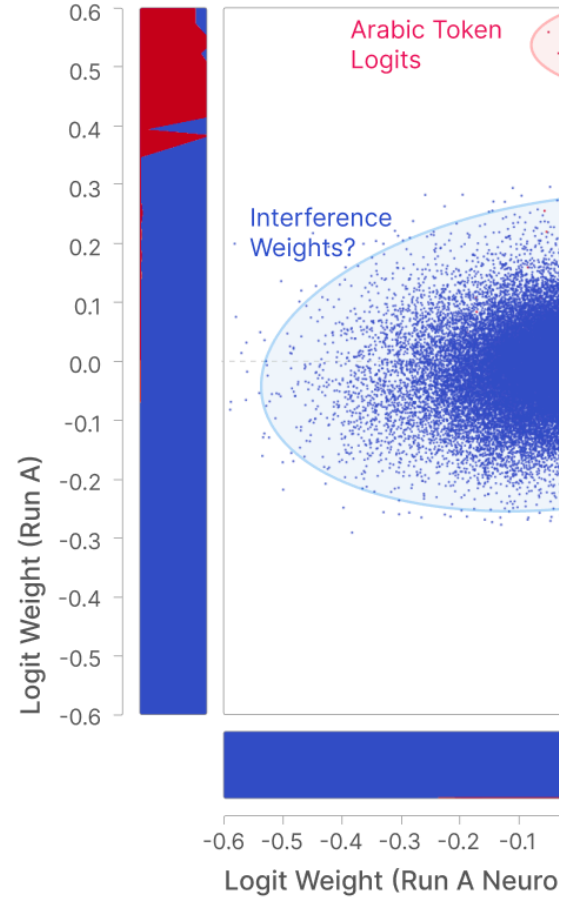
Feature Activations (A/1/3450 vs A/neuron/489)

Correlation between feature and neuron: 0.09



Logit Weights (A/1/3450 vs A/neuron/489)

Correlation between feature and

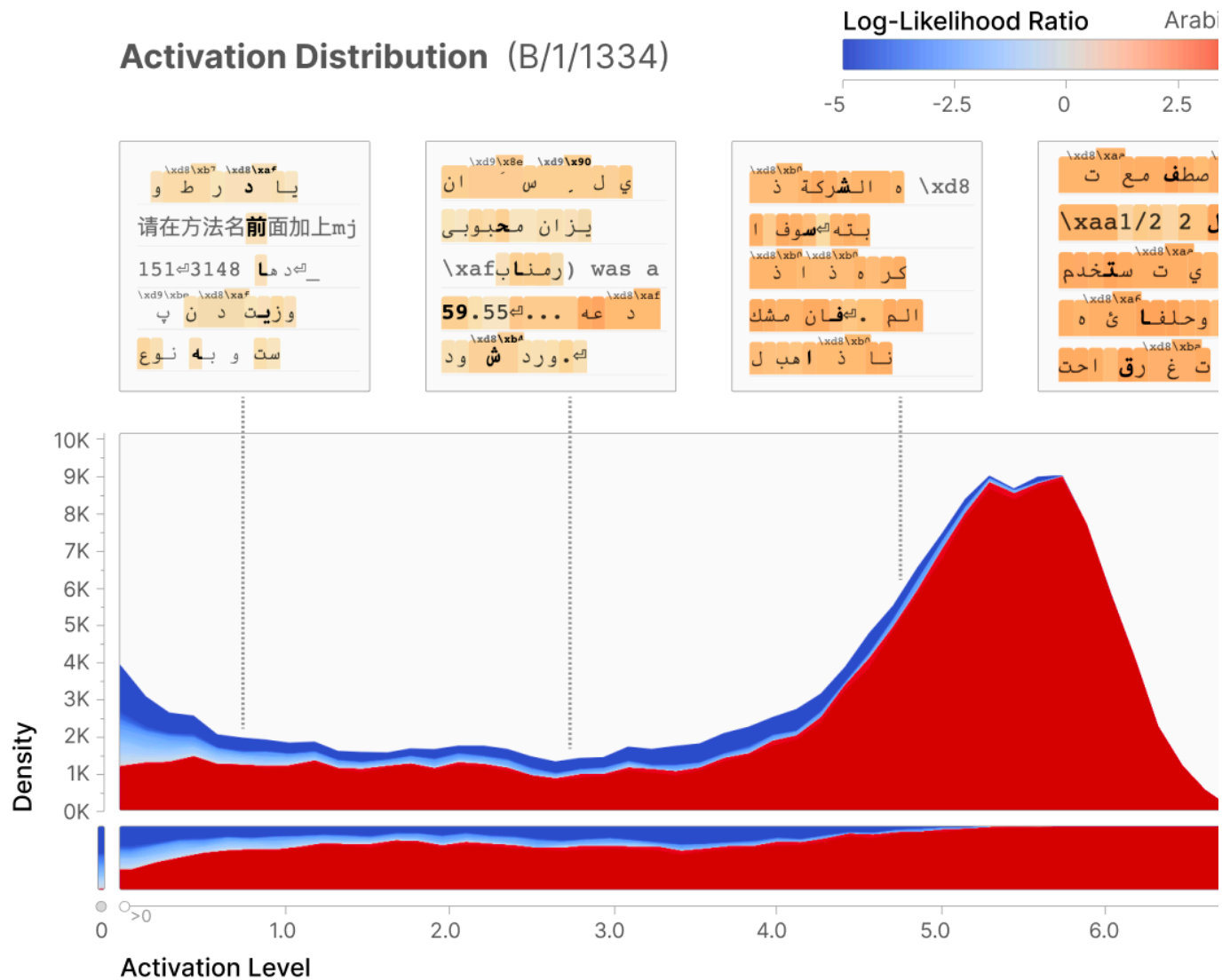


We conclude that the features we study do not trivially correspond to a single neuron. The Arabic script feature would be effectively invisible if we only analyzed the model in terms of neurons.

UNIVERSALITY

We will now ask whether A/1/3450 is a *universal* feature that forms in other models and can be consistently discovered by dictionary learning. This would indicate we are discovering something more general about how one-layer transformers learn representations of the dataset.

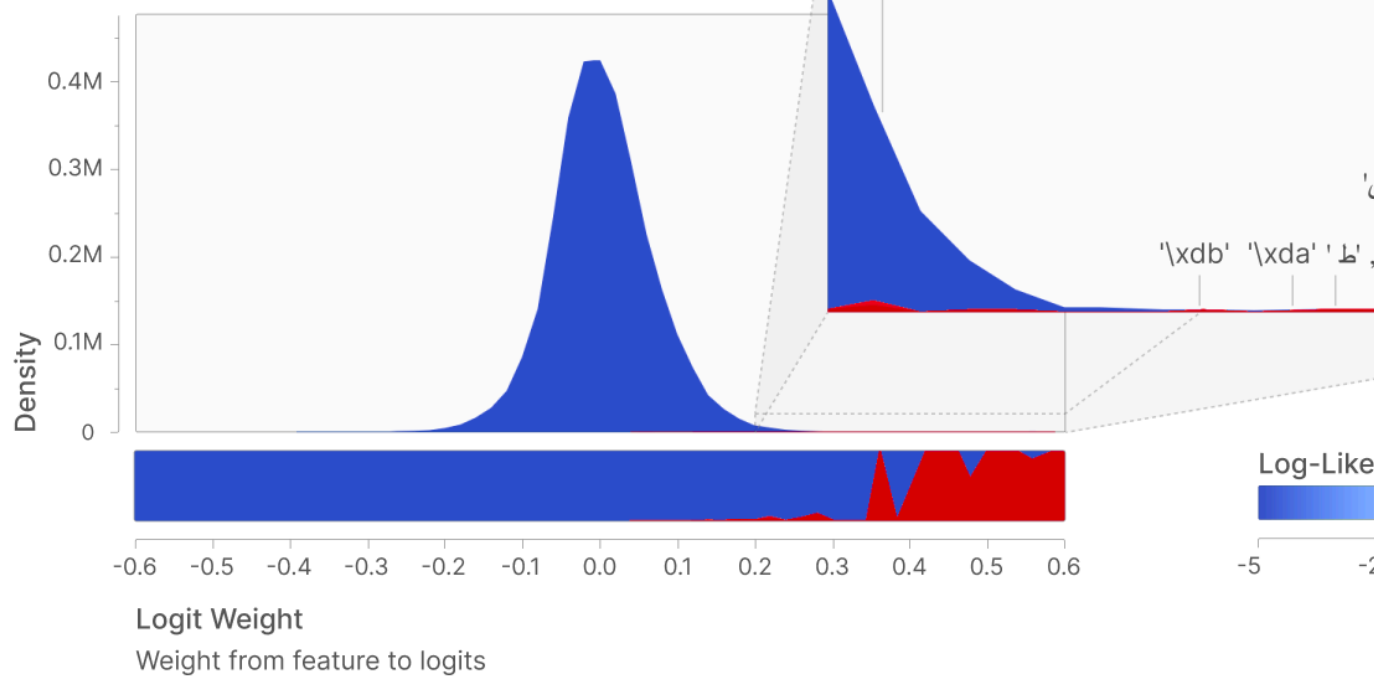
We search for a similar feature in B/1, a dictionary learning run on a transformer trained on the same dataset but with a different random seed. We search for the feature with the highest activation correlation²¹ and find B/1/1334 (corr=0.91), which is strikingly similar:



This feature clearly responds to Arabic script as well. If anything, it's nicer than our original feature – it's more specific in the 0–1 range. The logit weights tell a similar story:

Distribution of weights from feature to logit for each vocab element.

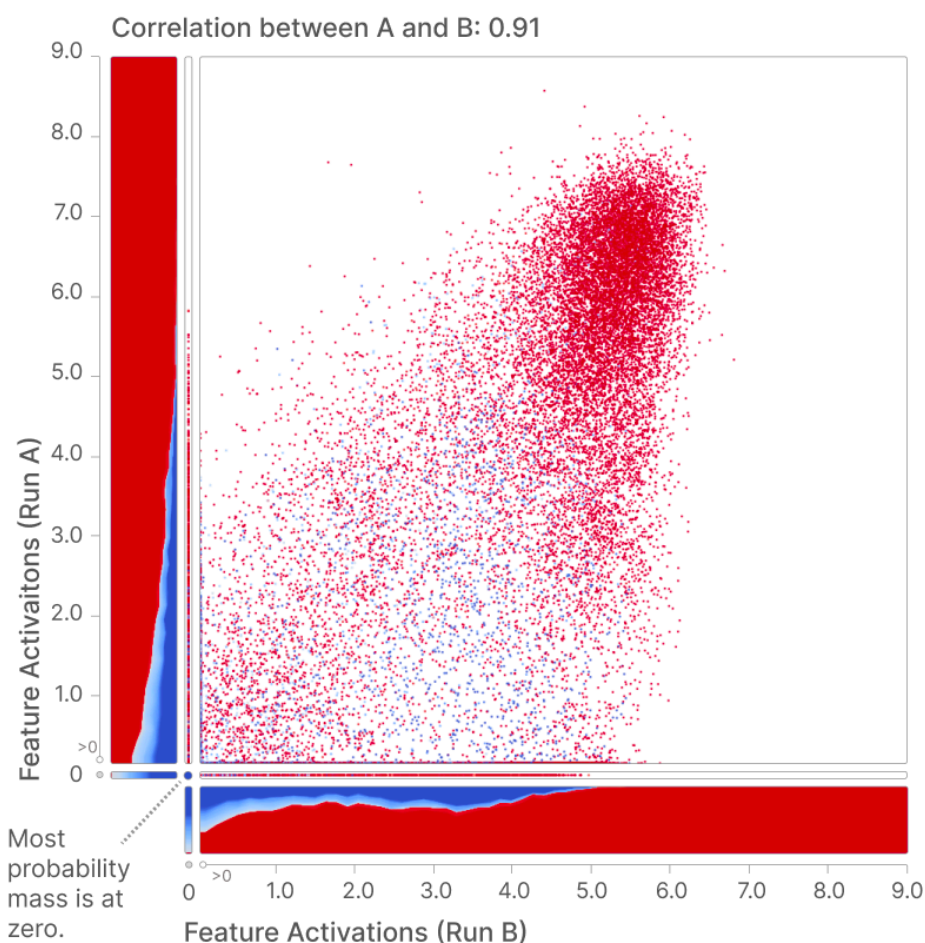
Distribution of weights from feature to logit for each vocab element.



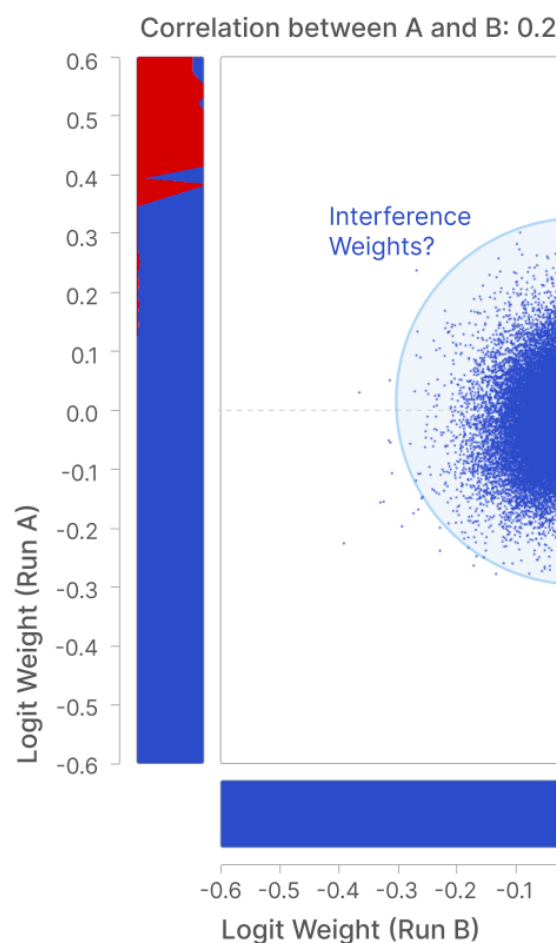
The effects of ablating this feature are also consistent with this (see the visualization for B/1/1334).

To more systematically analyze the similarities between A and B, we look at scatter plots comparing the activations or logit weights:

Feature Activations (A/1/3450 vs B/1/1334)



Logit Weights (A/1/3450 vs B/1/1334)



The activations are strongly correlated (Pearson correlation of 0.91), especially in the main Arabic mode.

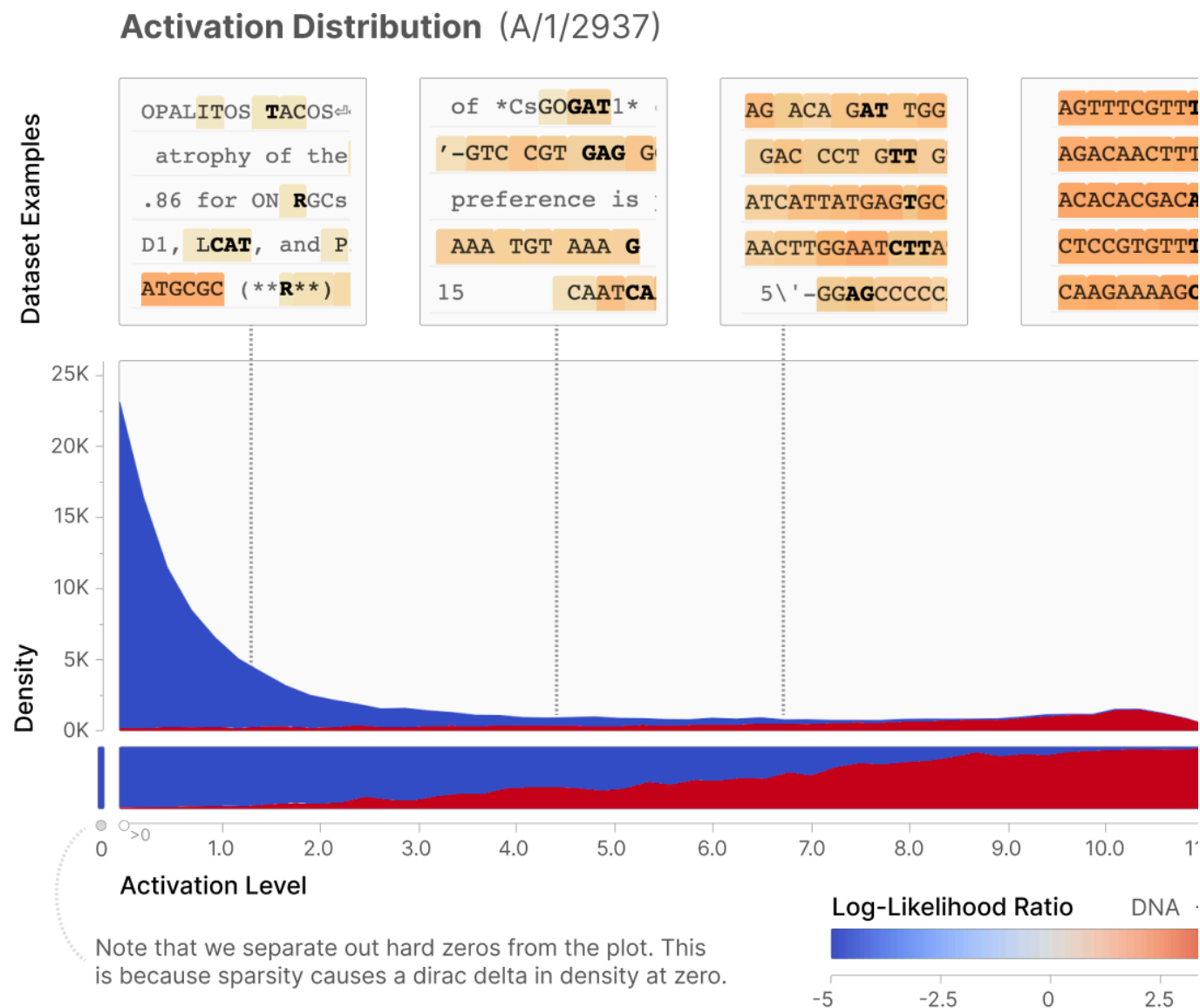
The logit weights reveal a two-dimensional version of the bimodality we saw in the histogram for A/1, with logit weights for Arabic tokens clustering at the top right. The correlation is more modest than that of the activations because the distribution is dominated by a relatively uncorrelated mode in the center. We hypothesize this central mode corresponds to "weight interference" and that the shared outlier mode is the important observation – that is, the model may ideally prefer to have all those weights be zero, but due to superposition with other features and their weights, this isn't possible.

DNA Feature

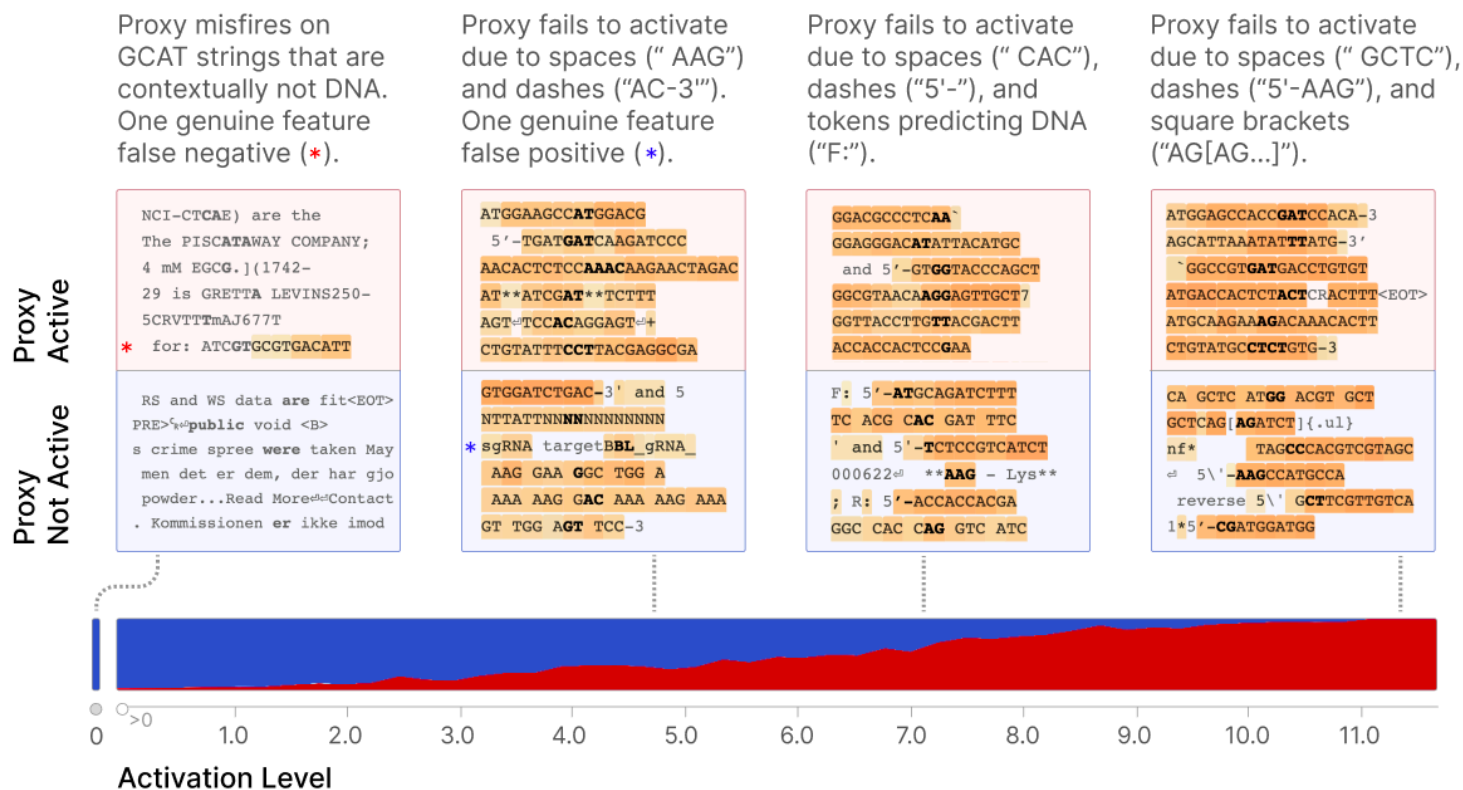
We now consider a DNA feature, A/1/2937. It activates in response to long uppercase strings consisting of A, T, C, and G, typically used to represent nucleotide sequences. We closely follow the analysis of the Arabic script feature above to show activation specificity and sensitivity for the feature, sensible downstream effects, a lack of neuron alignment, and universality between models. The main differences will be that (1) A/1/2937 is the only feature devoted to modeling the DNA context, and (2) our proxy is less sensitive to DNA than our feature is, missing strings containing punctuation, spaces, and missing bases.

ACTIVATION SPECIFICITY AND SENSITIVITY

We begin with the computational proxy for "is a DNA sequence", $\log(P(s|\text{DNA})/P(s))$. Because there are some vocabulary tokens, such as `CAT`, which could occur in DNA but also occur in other contexts, we always look at groups of at least two tokens when evaluating the proxy. The log-probabilities turn out to be quite bimodal, so we binarize the proxy (based on its sign). This binarized proxy then has a Pearson correlation of 0.8 with the feature activations.



While the feature appears to be quite monosemantic in the feature's higher registers (all 10 random dataset examples above activation of 6.0 are DNA sequences), there is significant blue indicating the DNA proxy not firing in the lower registers. Below we show a grid of random examples at four activation levels (including feature off) where the proxy does and doesn't fire.



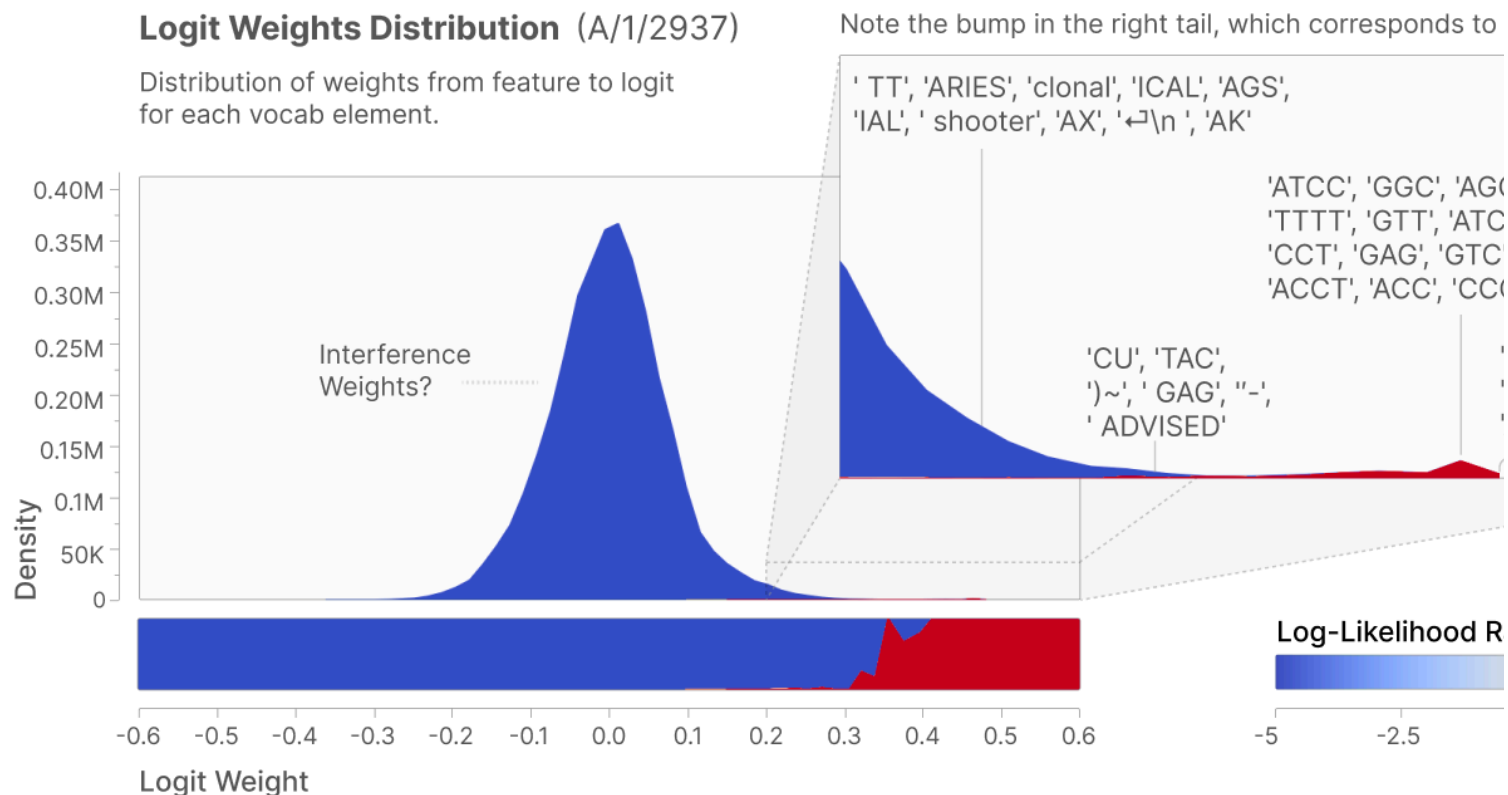
We note that in all but two cases where the feature and proxy disagree, the feature does indicate a DNA sequence, just one outside our proxy's strict `ATCG` vocabulary. For example, the space present in the triplets `TGG AGT` makes the proxy fail to fire. The feature also fires productively on `' -` in the string `5' -TCT`, because what follows that prefix should be DNA, even though the prefix is not itself DNA. (The causal ablation reveals that turning off the DNA feature hurts the prediction of the strings that follow.) We thus believe that [A/1/2937](#) is quite sensitive and specific for DNA. The case where the proxy fires and the feature does not is indeed a DNA sequence, though the feature begins firing on the very next token. We observe that the DNA feature may not fire on the first few tokens of a DNA sequence, but by the end of a long DNA sequence, it is the *only* feature active.

FEATURE DOWNSTREAM EFFECT

The downstream effect of the DNA feature being active, as measured by logit weights, make sense, with all the top tokens being combinations of nucleotides like `AGT` and `GCC`.

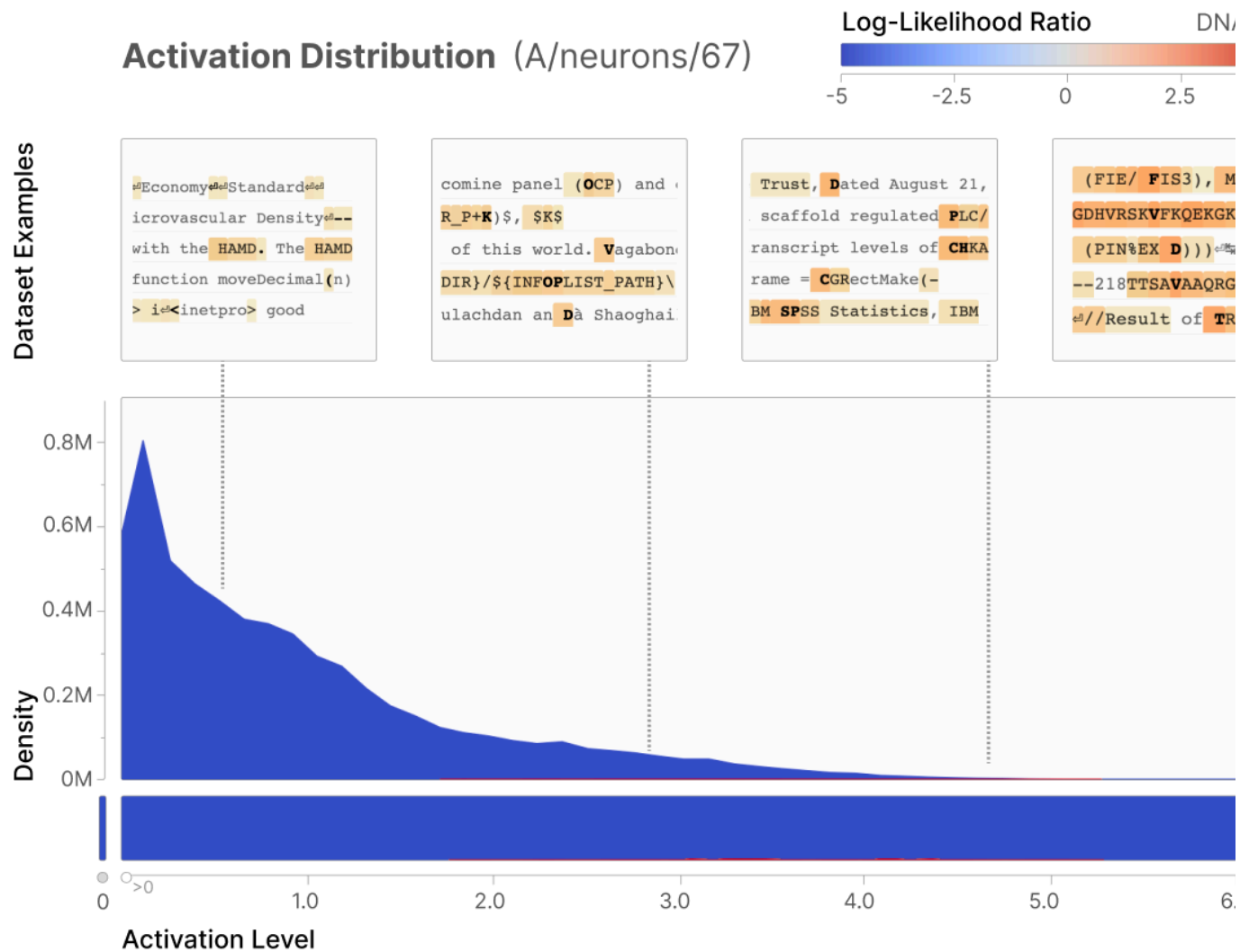
Logit Weights Distribution (A/1/2937)

Distribution of weights from feature to logit for each vocab element.



THE FEATURE IS NOT A NEURON

The most similar neuron to A/1/2937, as measured by activation correlation, is A/neurons/67. DNA contexts form a tiny sliver of that neuron's activating examples. The neuron whose coefficient is the highest in our feature's vector, A/neurons/227, also has no DNA sequences in its top activating examples.

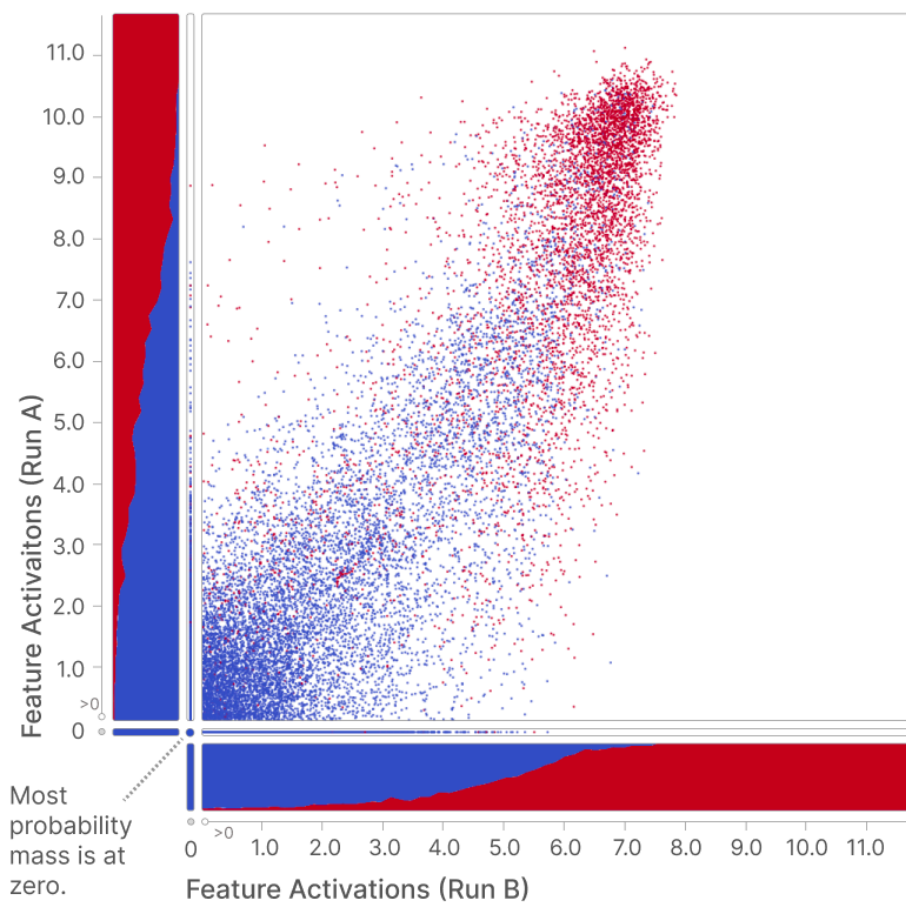


UNIVERSALITY

A/1/2937 has a correlated feature (corr=0.92) in run B/1, B/1/3680. Their top logit weights agree, and are DNA tokens (e.g. AGT) forming a separate mode (circled on the right) than the bulk of the logit weights for both.

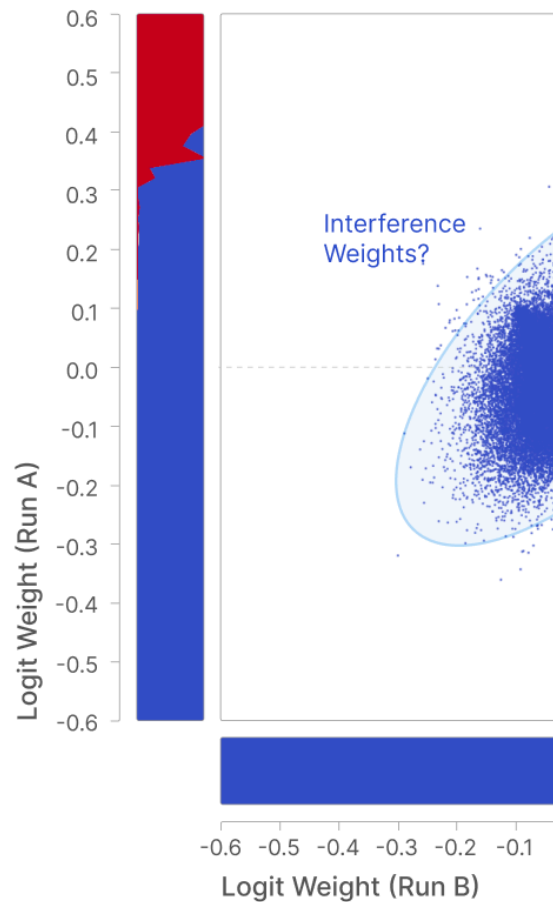
Feature Activations (A/1/2357 vs B/1/3680)

Correlation between A and B: 0.92



Logit Weights (A/1/2357

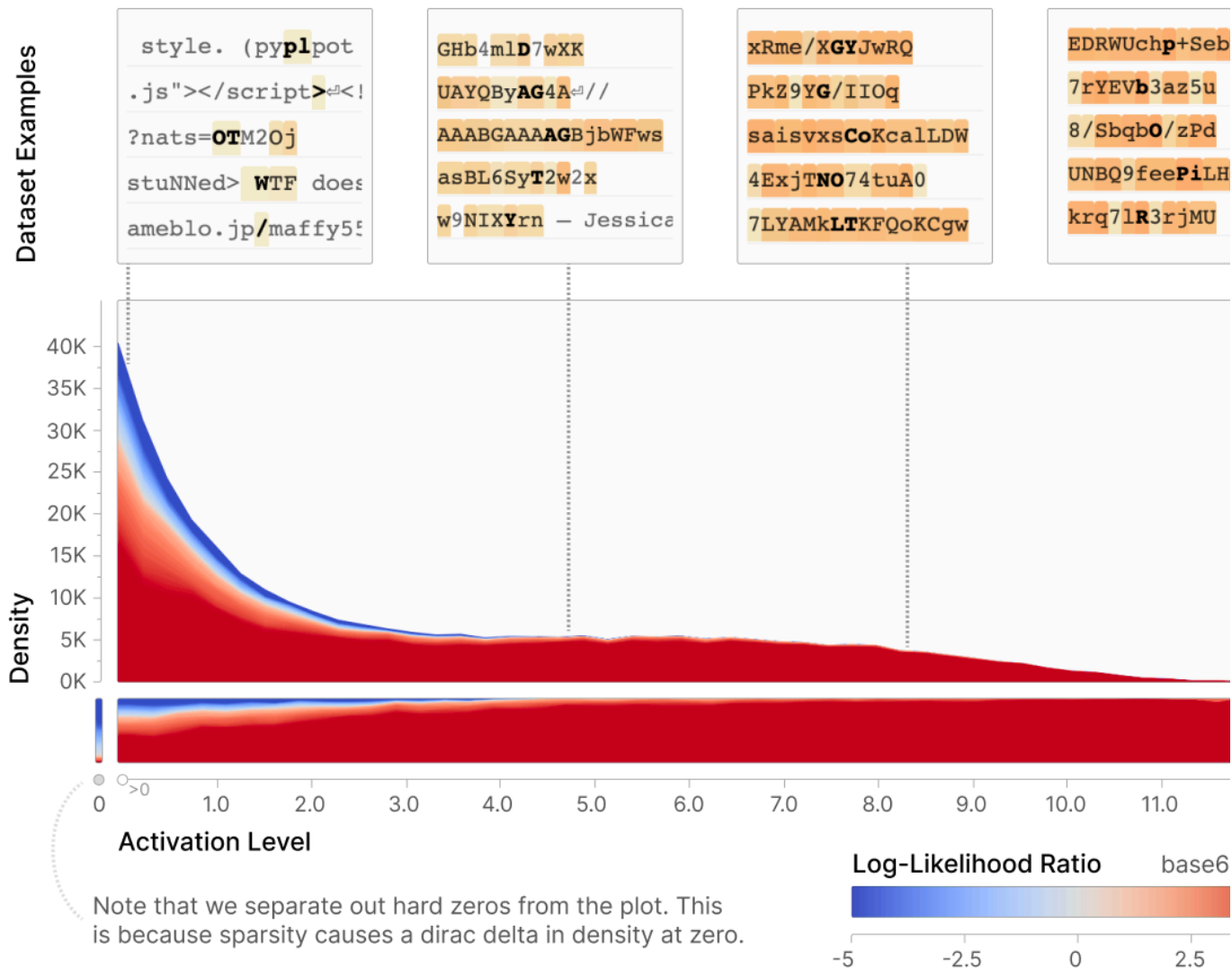
Correlation between A and B: 0.4



Base64 Feature

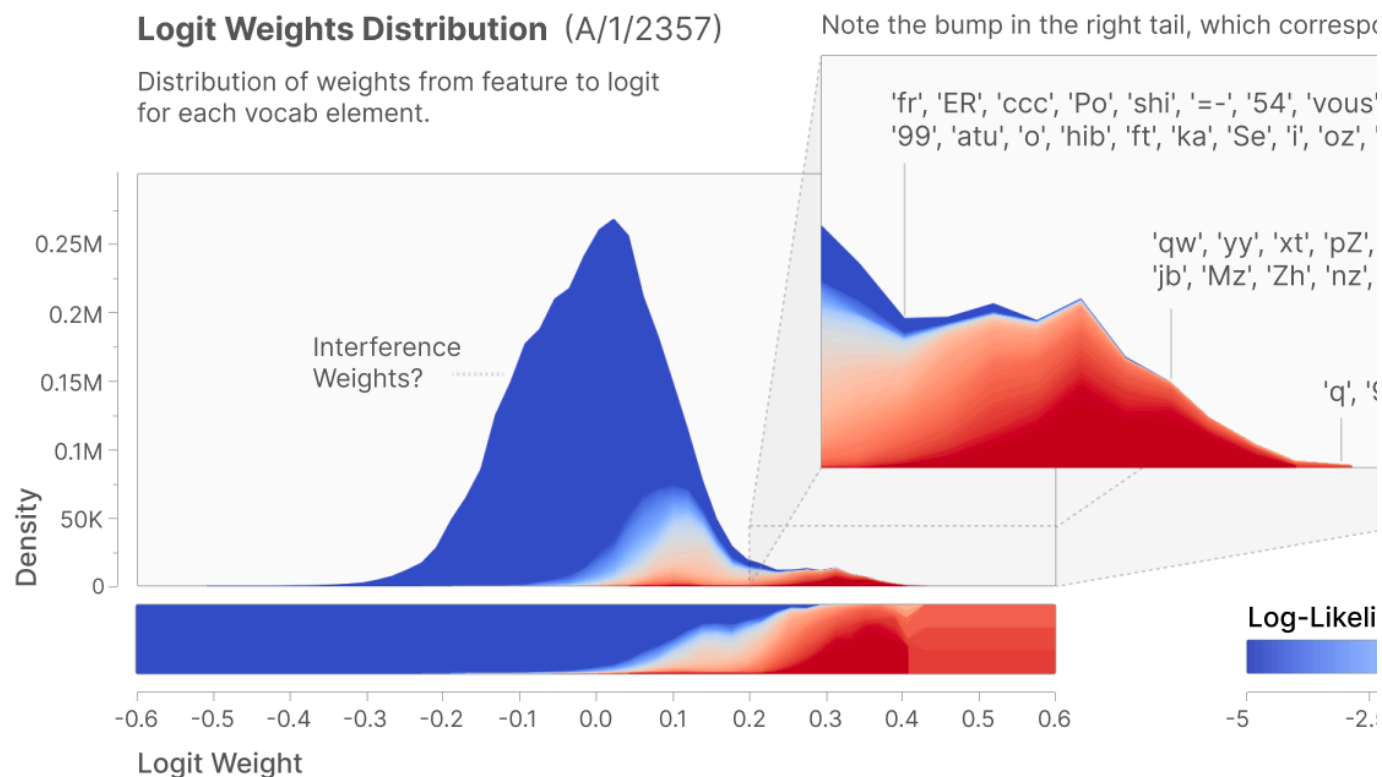
We now consider a base64 feature, A/1/2357. We're particularly excited by this feature because we discovered a base64 neuron in our SoLU paper [1], suggesting that base64 might be quite universal – even across models trained on somewhat different datasets and with different architectures. We model base64 strings as random sequences of characters from `[a-zA-Z0-9+/]`. The activation distribution colored by the corresponding computational proxy, together with the random dataset examples from each activation level, shows that this feature is quite specific to base64.

Activation Distribution (A/1/2357)



This is not the only feature active in base64 contexts, and in a section [below](#) we discuss the two others, one of which fires on single digits in base64 contexts (like the 2, 4, 7, and 9 on which A/1/2357 doesn't activate in the figure above), exploiting a property of the BPE tokenizer to make a better prediction.

Turning to the logit weights, they have a second mode consisting of highly base64-specific tokens. The main mode seems to primarily be interference, but the right side is skewed towards base64-neutral or slightly base64-leaning tokens. (If we look at the conditional below, we see a more continuous transition to base64-specific tokens.)

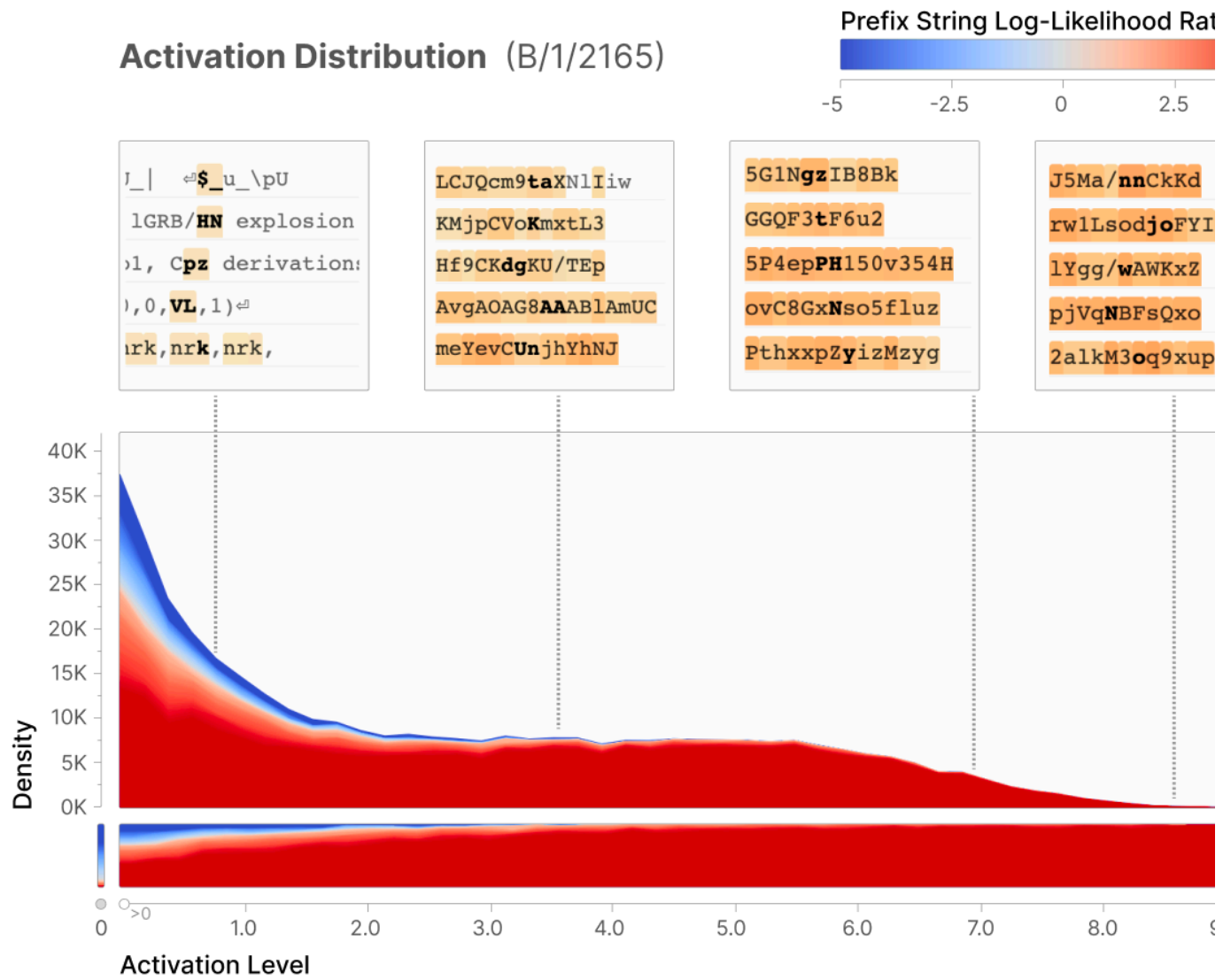


There is a more continuous transition between non-base64 and base64 tokens than we saw in the Arabic script example. This difference likely arises because whether a token occurs more in Arabic script than in other text is a relatively binary distinction, whereas whether a token occurs more in base64 or other text varies more continuously. For instance, `fr` is both a common abbreviation for the French language and also a base64 token, so it makes sense for the model to be cautious in up-weighting `fr` because it might already have a higher prior due to use in French. Indeed, any token consisting of letters from the English alphabet will have some nontrivial probability of appearing in base64 strings.

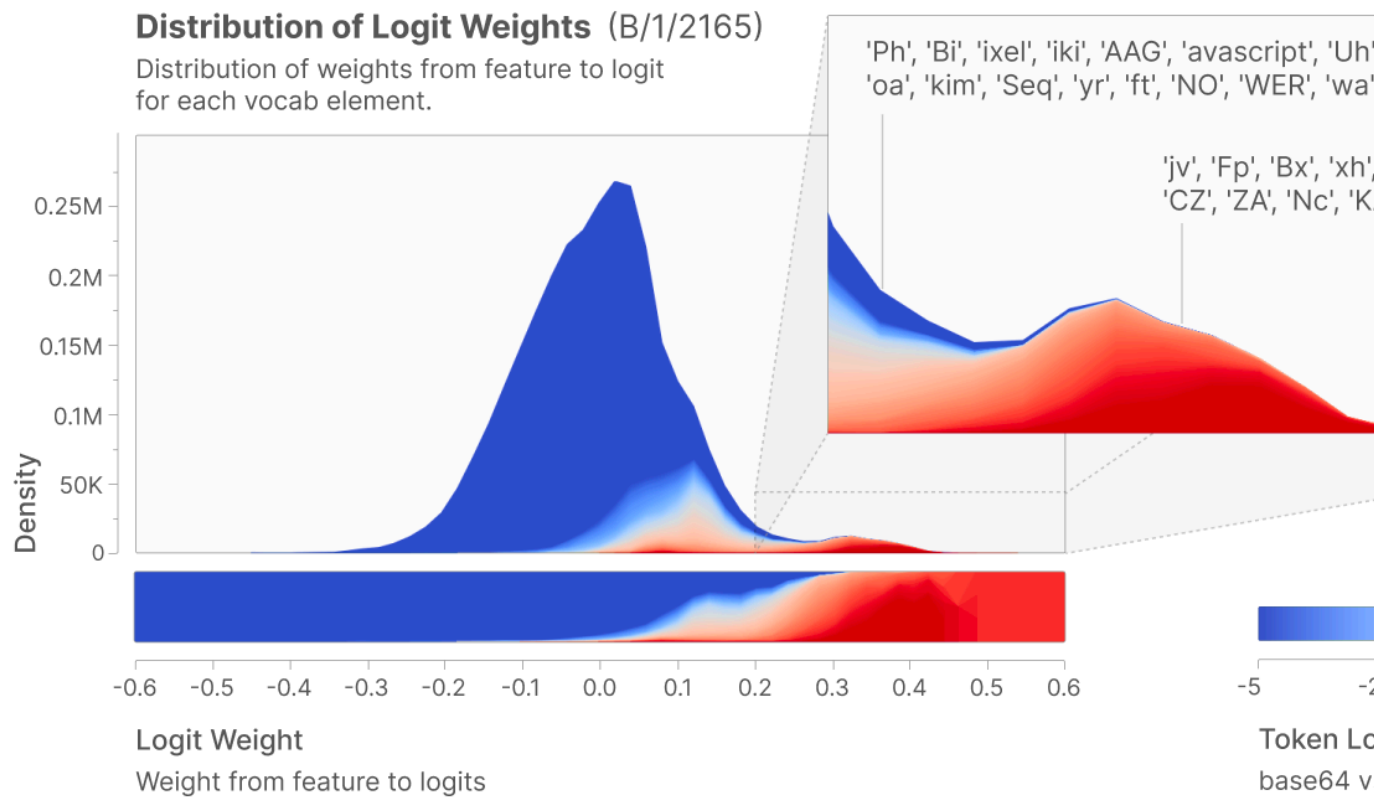
The Pearson correlation between the computational proxy and the activity of A/1/2357 is just 0.38. We believe that is mostly because the proxy is too broad. For example hexadecimal strings (those made of `[0-9A-F]`) activate the proxy, as they are quite different from the overall data distribution, but are actually predicted by a feature of their own, A/1/3817.

UNIVERSALITY

A/1/2357 has a correlated feature (corr=0.85) in run B/1, B/1/2165. It also has high activation specificity for base64 strings:



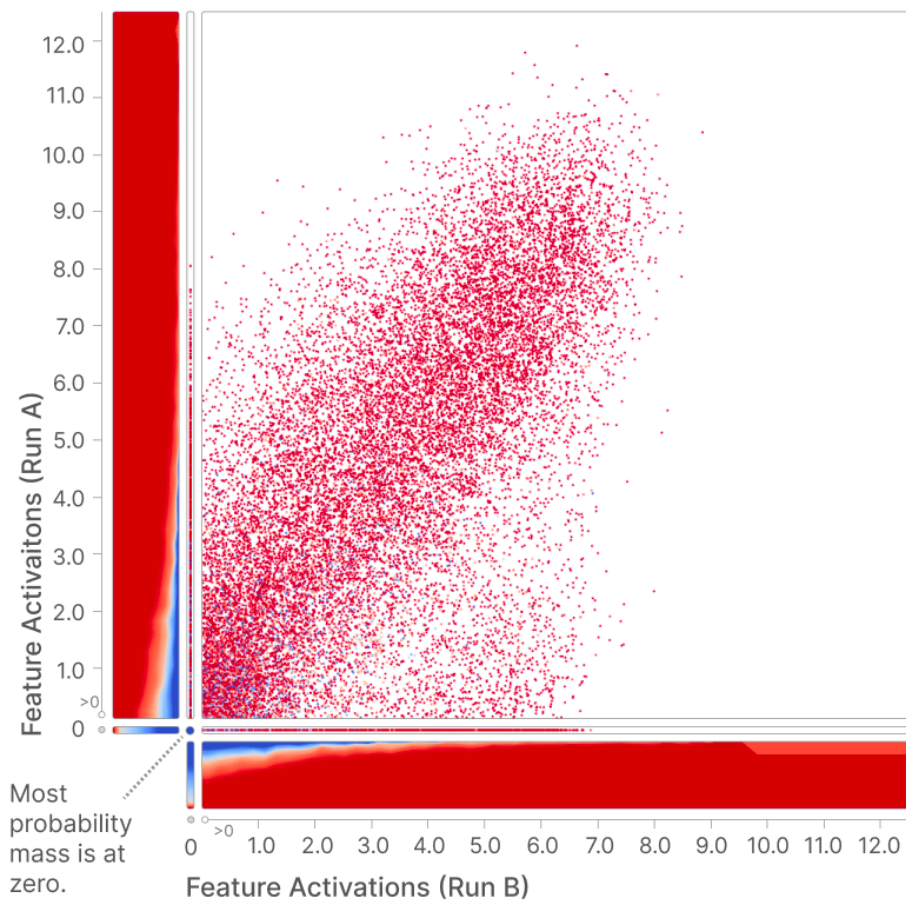
Like [A/1/2357](#), [B/1/2165](#)'s logit weights have a second mode corresponding to base64 tokens:



Correlations and scatter plots are also consistent with them being very similar features:

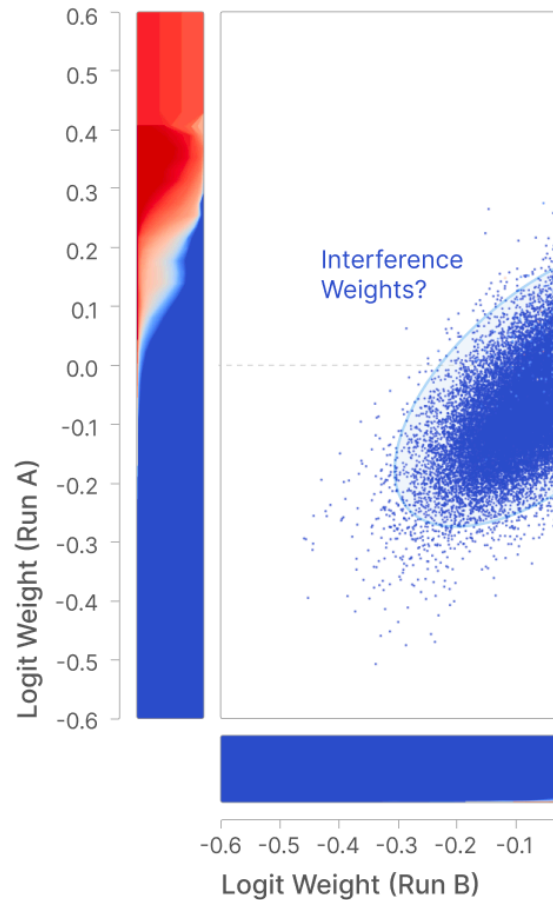
Feature Activations (A/1/2357 vs B/1/2165)

Correlation between A and B: 0.85



Logit Weights (A/1/2357 vs B/1/2165)

Correlation between A and B: 0.8



Note that we expect the overlap between the interference and base64 token logit weights to be from the aforementioned usage of base64 tokens across many other contexts.

THE FEATURE IS NOT A NEURON

Looking at the neuron in model A that most correlates with this feature: [A/neurons/470](#) (corr=0.18), we find that while it does notably respond to base64 strings, it also activates for lots of other things, including code, HTML labels, parts of URLs, etc.: